



UNIVERSITÉ DE LIMOGES

FACULTÉ DES SCIENCES ET TECHNIQUES

Projet d'initiation à la recherche

Cryptanalyse
d'implémentations RSA : petits
exposants secrets et oracles
PKCS#1 v1.5

Réalisé par :

Karim Mohamed Cherif RAHMANI
Mamadou Taifour BALDE

Encadré par :

Mercedes HAIECH

Année universitaire : 2025-2026

Table des matières

1	Principe du cryptosystème RSA	4
1.1	Définitions et théorèmes	4
1.2	Cryptosystème RSA	6
1.2.1	Génération des clés	6
1.2.2	Chiffrement et déchiffrement	7
1.2.3	Preuve de correction RSA	9
1.3	Optimisation par le théorème des restes chinois (CRT)	12
2	Paramètres de sécurité de RSA	14
2.1	Choix des nombres premiers p et q	14
2.1.1	Taille de p et q	14
2.1.2	Proximité	15
2.2	Choix de l'exposant public	16
2.2.1	Danger de l'exposant publique trop petit	16
2.2.2	Recommandation pour le choix de l'exposant public e	18
2.2.3	Risques liées a un exposant d très petit	18
3	Attaque de Wiener	19
3.1	Les fractions continues	19
3.1.1	Le développement en fraction continue	20
3.1.2	Calcul des convergentes	20
3.2	Principe mathématique de l'attaque de Wiener	21
3.3	Conditions de réussite de l'attaque	22
3.4	Algorithme de Wiener	23
3.4.1	Développement en fraction continue	23
3.4.2	Calcul des convergentes	24
3.4.3	Vérification des candidats	24
4	Le standard de chiffrement PKCS#1 v1.5	26
4.1	Le format PKCS1 v1.5	27
4.1.1	Le chiffrement	27
4.1.2	Le déchiffrement	29
4.2	L'Oracle de conformité PKCS#1 v1.5	32

4.2.1	Définition de l'Oracle	33
4.2.2	Modélisation Mathématique	33
5	Protocole d'attaque de Bleichenbacher	35
5.1	Phase 1 : Le masquage (Blinding)	35
5.2	Phase 2 : Recherche de messages conformes	37
5.3	Phase 3 : Réduction de l'ensemble des intervalles	40
5.4	Phase 4 : Extraction du message clair	42
.1	Implémentation d'une attaque naïve et l'attaque de Wiener	45
.2	Résultat d'application du code	48
.3	Implémentation de l'attaque par oracle	49
.4	Résultat de l'attaque par oracle	54

Introduction

Le cryptosystème RSA est l'un des schémas à clé publique les plus importants de l'histoire de la cryptographie. Il peut être employé pour chiffrer et générer des signatures numériques, pour assurer l'authenticité des communications ou des données. Aujourd'hui, il reste largement employé, en particulier pour la signature, même si d'autres mécanismes sont souvent privilégiés pour l'échange de clés, notamment ECDHE et X25519, qui reposent sur l'arithmétique des courbes elliptiques, ainsi que X3DH et PQXDH, qui combinent eux aussi des échanges de type Diffie–Hellman sur courbes elliptiques et, dans le cas de PQXDH, une composante post-quantique fondée sur les réseaux euclidiens via le problème Module Learning with Errors. La sécurité de RSA repose sur la difficulté de factoriser un grand nombre entier, produit de deux nombres premiers de grande taille. On commence par rappeler les notions algébriques nécessaires, puis on décrit en détail le cryptosystème RSA. On s'intéresse ensuite à l'optimisation du déchiffrement par le théorème des restes chinois, avant d'étudier l'impact du choix des paramètres de sécurité.

Une attention particulière est portée à l'attaque de Wiener, qui montre qu'un exposant secret trop petit rend le système vulnérable, et dont le principe repose sur les fractions continues et leurs convergentes pour retrouver la clé privée à partir de la clé publique.

Cependant, la solidité mathématique du protocole RSA brut (*Textbook RSA*) s'avère insuffisante pour garantir la sécurité des implémentations en conditions réelles. Par conséquent, nous élargissons l'analyse aux vulnérabilités liées à l'implémentation des standards de cryptographie, en nous concentrant sur le standard PKCS#1 v1.5. Bien qu'introduit pour pallier les limites du RSA scolaire grâce à un schéma de remplissage probabiliste (*padding*), ce standard peut générer un « oracle de conformité » si les erreurs de déchiffrement sont mal gérées. C'est précisément l'exploitation de cet oracle qui constitue le cœur de l'attaque de Bleichenbacher (1998), à laquelle aboutit ce projet. Nous y décrirons pas à pas comment cette redoutable attaque adaptative permet, via une réduction itérative d'intervalles, de retrouver le message clair sans jamais compromettre la clé privée, soulignant ainsi l'importance cruciale d'une implémentation rigoureuse au-delà de la seule théorie mathématique.

Chapitre 1

Principe du cryptosystème RSA

Un cryptosystème a clef publique aussi appelé chiffrement asymétrique est un système de chiffrement dont la sécurité ne repose pas sur la connaissance de la clef de chiffrement. C'est un système où la clé publique du destinataire qui permet de chiffrer est connue de tous, tandis que sa clef privée, qui permet de déchiffrer les messages reste secrète. Ce qui évite de devoir communiquer la clef sur un canal sécurisé avant de commencer à communiquer.

La première apparition d'un tel système était en 1976, le cryptosystème RSA (nommé d'après ses inventeurs, Rivest, Shamir et Adleman) cette invention était une révolution en cryptographie. Sa sécurité repose sur la difficulté de factoriser des grands nombres.

1.1 Définitions et théorèmes

Définition 1.1.1. On appelle nombre premier tout entier $p \geq 2$ dont les seuls diviseurs positifs sont 1 et p .

Définition 1.1.2. PGCD : le plus grand diviseur en commun

Soit a et $b \in \mathbb{N}$ on dit que $\gcd(a, b) = d$ si :
 d est le plus grand nombre qui divise a et b

Définition 1.1.3. On dit que a et b sont premiers entre eux si $\gcd(a, b) = 1$

Définition 1.1.4. On appelle un anneau l'ensemble A muni de deux lois de composition interne $(A, +, \times)$ tel que :

1. $(A, +)$ est un groupe commutatif
2. la loi $(\times)$ est associative, distributive par rapport à la loi $(+)$, et elle admet un élément neutre sur A :
Soit $a, b, c \in A$:

Associative $a \times (b \times c) = (a \times b) \times c$.

Distributive $a \times (b + c) = (a \times b) + (a \times c)$
 $(b + c) \times a = (b \times a) + (c \times a)$

Remarque :

$(\mathbb{A}, +, \times)$ est un anneau commutatif si la loi $(\times)$ est commutative : $a \times b = b \times a$
 $(\mathbb{A}, +, \times)$ est un anneau unitaire (appelé aussi unifié) s'il existe un élément neutre pour la loi $(\times)$: $1_A \times a = a \times 1_A = a$

Définition 1.1.5. L'indicatrice d'Euler φ est une fonction qui pour tout entier naturel non nul n associe le nombre d'entiers strictement inférieurs à n qui sont premiers avec n .

$$\varphi(n) = \text{card}\{k \in \mathbb{N} \mid 1 \leq k < n \text{ et } \text{pgcd}(n, k) = 1\} \quad (1.1)$$

On a pour tout nombre premier p , $\varphi(p) = p - 1$ car tout les nombre non nul inférieur a p sont premiers avec p .

Donc pour tout $r \in \mathbb{N}^*$ on a :

$$\varphi(p^r) = \text{card}\{k \in \mathbb{N} \mid 1 \leq k < p^r \text{ et } \text{pgcd}(p^r, k) = 1\}$$

Les nombres qui divisent p^r sont les multiples de p : $\{p, 2 \times p, 3 \times p, \dots, p^{r-1} \times p\}$

Il y a donc exactement p^{r-1} diviseurs de p^r

$$\text{Donc } \varphi(p^r) = p^r - p^{r-1}$$

On sait que pour tout $n \in \mathbb{N}$ on peut factoriser n comme suit : $n = \prod p_i^{\alpha_i}$ tel que p_i sont les nombres premiers diviseurs de n et $\alpha_i \in \mathbb{N}$, ce qui nous donne :

$$\varphi(n) = \varphi(\prod p_i^{\alpha_i}) = \prod \varphi(p_i^{\alpha_i}) = \prod (p_i^{\alpha_i} - p_i^{\alpha_i-1})$$

Définition 1.1.6. Congruence modulo un entier

La divisibilité permet de définir une relation d'équivalence sur les entiers, la congruence modulo un entier.

Soit $n \in \mathbb{N}$ et soient $a, b \in \mathbb{Z}$ on dit que a congru a b modulo n si $n \mid (a - b)$.

On note alors :

$$a \equiv b \pmod{n}$$

Définition 1.1.7. Pour tout $n \geq 1$ l'anneau quotient $\mathbb{Z}/n\mathbb{Z}$ est l'ensemble des classes d'équivalence de $\mathbb{Z}$ pour la relation de congruence modulo n muni des opérations d'addition et de multiplication héritées de $\mathbb{Z}$

$$\mathbb{Z}/n\mathbb{Z} = \{0, 1, \dots, n-1\} \quad (1.2)$$

Théorème 1.1.1. Soit $n \in \mathbb{N}^*$. Les éléments inversibles de $\mathbb{Z}/n\mathbb{Z}$ Sont les classes $\bar{a}$ tel que $\text{gcd}(a, n) = 1$, de sorte que $(\mathbb{Z}/n\mathbb{Z})^\times$ est un groupe multiplicatif d'ordre $\varphi(n)$.

$$(\mathbb{Z}/n\mathbb{Z})^\times = \{k \in \{0, \dots, n-1\} \mid \text{gcd}(k, n) = 1\} \quad (1.3)$$

En particulier $\mathbb{Z}/n\mathbb{Z}$ est un corps si et seulement si n est un nombre premier. c'est à dire :

$$(\mathbb{Z}/n\mathbb{Z})^\times = \mathbb{Z}/n\mathbb{Z} \setminus \{0\}$$

Remarque 1.1.1. Pour tout $x \in (\mathbb{Z}/n\mathbb{Z})^\times$, il existe $x^{-1} \in (\mathbb{Z}/n\mathbb{Z})^\times$ tel que $x \times x^{-1} \equiv 1 \pmod{n}$. Pour calculer x^{-1} on utilise l'algorithme d'euclide étendu.

Algorithme d'Euclide étendu

Étant donnés deux entiers a et b , il calcule leur plus grand commun diviseur $d = \gcd(a, b)$ ainsi qu'un couple de coefficients de Bézout $(u, v) \in \mathbb{Z}^2$ vérifiant :

$$au + bv = d \tag{1.4}$$

Algorithme 1 : Algorithme d'Euclide étendu

Entrées : Deux entiers a et b (avec $a \geq 0, b \geq 0$)

Sorties : Un triplet (d, u, v) tel que $au + bv = d = \gcd(a, b)$

$u \leftarrow 1, \quad u' \leftarrow 0;$

$v \leftarrow 0, \quad v' \leftarrow 1;$

tant que $b \neq 0$ **faire**

$q \leftarrow \lfloor a/b \rfloor$
 $(a, b) \leftarrow (b, a - q \times b);$
 $(u, u') \leftarrow (u', u - q \times u');$
 $(v, v') \leftarrow (v', v - q \times v');$

retourner (a, u, v)

1.2 Cryptosystème RSA

1.2.1 Génération des clés

Cette étape consiste à générer une paire de clé : une clé publique, qui peut être partagée avec n'importe qui, et une clé privée, qui doit rester secrète. La génération se déroule comme suit :

1. **Choisir deux nombres premiers :** On choisit deux grands nombres premiers distincts, p et q .
Pour une sécurité moderne, p et q sont choisis de telle sorte que leur produit n ait une taille de 2048, 3072 ou 4096 bits.
2. **Calculer le module :** On calcule leur produit, $n = pq$. Ce nombre n est le module de chiffrement et fera partie des deux clés.
Toutes les opérations de chiffrement et de déchiffrement seront effectuées **modulo** n .

3. **L'indicatrice d'Euler** : On a $n = p \times q$ avec p et q des nombres premiers donc :

$$\varphi(n) = (p-1)(q-1) \quad (1.5)$$

4. **Choisir l'exposant public** : On choisit un entier e tel que $1 < e < \varphi(n)$ et $\gcd(e, \varphi(n)) = 1$.
5. **Calculer l'exposant privé** : On calcule d qui est l'inverse de e modulo $\varphi(n)$ en utilisant l'algorithme d'euclide étendu :

$$d \equiv e^{-1} \pmod{\varphi(n)}$$

Cela signifie que d est l'unique entier (modulo $\varphi(n)$) tel que

$$ed \equiv 1 \pmod{\varphi(n)}$$

Après avoir suivis toutes les étapes on a nos deux clés :

Clé Publique : (n, e) .

Clé Privée : (d, p, q) .

1.2.2 Chiffrement et déchiffrement

Une fois les clés générées, le processus pour chiffrer et déchiffrer un message m est très direct.

Définition 1.2.1 (Fonction de chiffrement RSA). Soit (n, e) une clé publique RSA. La fonction de chiffrement, notée E , est l'application définie de l'anneau $\mathbb{Z}/n\mathbb{Z}$ dans lui-même par :

$$\begin{aligned} \mathcal{E} : \mathbb{Z}/n\mathbb{Z} &\rightarrow \mathbb{Z}/n\mathbb{Z} \\ m &\mapsto m^e \pmod{n} \end{aligned}$$

Pour un message clair $m \in \mathbb{Z}/n\mathbb{Z}$, le chiffré est l'image $c = \mathcal{E}(m)$.

Définition 1.2.2 (Fonction de déchiffrement RSA). Soit (n, d) la clé privée RSA associée. La fonction de déchiffrement, notée $\mathcal{D}$, est l'application définie de l'anneau $\mathbb{Z}/n\mathbb{Z}$ dans lui-même par :

$$\begin{aligned} \mathcal{D} : \mathbb{Z}/n\mathbb{Z} &\rightarrow \mathbb{Z}/n\mathbb{Z} \\ c &\mapsto c^d \pmod{n} \end{aligned}$$

Pour un message chiffré $c \in \mathbb{Z}/n\mathbb{Z}$, le message clair retrouvé est l'image $m = \mathcal{D}(c)$.

Exemple 1.2.1. Pour illustrer le fonctionnement du cryptosystème RSA, simulons un échange avec des paramètres volontairement petits.

1. Génération des clés Supposons que le destinataire (Alice) choisisse les nombres premiers suivants :

— $p = 17$ et $q = 11$.

Elle calcule le module de chiffrement n et l'indicatrice d'Euler $\varphi(n)$:

$$n = p \times q = 17 \times 11 = 187$$

$$\varphi(n) = (p - 1)(q - 1) = 16 \times 10 = 160$$

Elle choisit un exposant public e tel que $\gcd(e, 160) = 1$ et $1 < e < 160$. Prenons :

$$e = 3$$

Elle calcule ensuite son exposant privé d , l'inverse de e modulo $\varphi(n)$. Elle cherche d tel que $ed \equiv 1 \pmod{160}$. En utilisant l'algorithme d'Euclide étendu on retrouve

$$d = 107$$

Les clés sont donc :

- **Clé Publique** ($\mathbf{n}, \mathbf{e}$) : $(187, 3)$
- **Clé Privée** ($\mathbf{p}, \mathbf{q}, \mathbf{d}$) : $(17, 11, 107)$

2. Chiffrement du message Un expéditeur (Bob) souhaite envoyer le message $m = 15$ à Alice. Il utilise la clé publique $(187, 3)$ et la fonction de chiffrement $\mathcal{E}$:

$$c = \mathcal{E}(m) = m^e \pmod{n}$$

$$c \equiv 15^3 \pmod{187}$$

$$c \equiv 3375 \pmod{187}$$

$$c = 9 \pmod{187}$$

Bob envoie le chiffré $c = 9$ à Alice.

3. Déchiffrement du message Alice reçoit $c = 9$. Elle utilise sa clé privée $(17, 11, 107)$ et la fonction de déchiffrement $\mathcal{D}$:

$$m = \mathcal{D}(c) = c^d \pmod{n}$$

$$m \equiv 9^{107} \pmod{187}$$

$$m \equiv 15 \pmod{187}$$

Alice retrouve bien le message original $m = 15$.

1.2.3 Preuve de correction RSA

Le terme **correction** utilisé pour désigner un algorithme cryptographique ne fait pas allusion à sa sécurité, mais à sa capacité déterministe à inverser le processus de chiffrement pour retrouver le message original sans perte ni altération.

Pour le système RSA prouver sa correction revient à prouver ce qui suit :

$$\begin{aligned}\mathcal{D}(\mathcal{E}(m)) &= \mathcal{D}(c) \\ &= (m^e)^d \\ &= m\end{aligned}$$

Donc prouver la correction revient à démontrer l'identité arithmétique suivante :

$$(m^e)^d \equiv m \pmod{n}, \forall m \in \mathbb{Z}/n\mathbb{Z} \quad (1.6)$$

Avant d'établir la preuve, nous rappelons quelques notions fondamentales.

Théorème 1.2.1 (Théorème de Lagrange). *Soit G un groupe fini d'ordre n . Pour tout élément a appartenant à G , l'ordre de a divise l'ordre du groupe G .*

Si m est l'ordre du groupe G et e_G l'élément neutre du groupe, alors pour tout $a \in G$:

$$a^n = e_G \quad (1.7)$$

Corollaire 1.2.1. Soient p et q deux nombres premiers distincts et $n = pq$. Alors on a l'isomorphisme suivant :

$$\begin{aligned}(\mathbb{Z}/n\mathbb{Z})^\times &\simeq (\mathbb{Z}/p\mathbb{Z})^\times \times (\mathbb{Z}/q\mathbb{Z})^\times \\ x \pmod{n} &\mapsto (x \pmod{p}, x \pmod{q})\end{aligned}$$

L'ordre du groupe $(\mathbb{Z}/n\mathbb{Z})^\times$ est donné par $\varphi(n) = (p-1)(q-1)$.

Alors pour tout entier a tel que $\gcd(a, n) = 1$ on a :

$$a^{\varphi(n)} \equiv 1 \pmod{n} \quad (1.8)$$

Théorème 1.2.2. Le petit théorème de Fermat

Soit p un nombre premier, et $(\mathbb{Z}/p\mathbb{Z})^\times$ un groupe cyclique d'ordre $\varphi(p) = p-1$. Alors, On a l'isomorphisme suivant :

$$((\mathbb{Z}/p\mathbb{Z})^\times, \times) \simeq (\mathbb{Z}/(p-1)\mathbb{Z}, +)$$

Alors, pour tout entier a on a la congruence suivante :

$$a^{p-1} \equiv 1 \pmod{p}$$

Démonstration 1.2.1.

Le groupe $(\mathbb{Z}/p\mathbb{Z})^\times$ est d'ordre $p-1$, D'après le théorème de Lagrange, l'ordre de tout élément divise l'ordre du groupe. Par conséquent, pour tout entier $a \in (\mathbb{Z}/p\mathbb{Z})^\times$

$$a^{p-1} \equiv 1 \pmod{p}$$

Théorème 1.2.3. Theoreme des restes chinois

Soient p et q deux entiers strictement positifs et premiers entre eux ($\gcd(p, q) = 1$) et n leurs produits ($n = pq$). Alors l'application :

$$\begin{array}{ccc} \mathbb{Z}/n\mathbb{Z} & \simeq & \mathbb{Z}/p\mathbb{Z} \times \mathbb{Z}/q\mathbb{Z} \\ x \bmod n & \mapsto & (x \bmod p, x \bmod q) \end{array}$$

est un isomorphisme d'anneau.

On peut présenter ce théorème sous une forme plus élémentaire en utilisant le système des congruences.

Pour p et q premiers entre eux, et pour tout couple d'entiers (a, b) . Alors le système de congruences :

$$\begin{cases} x \equiv a \pmod{p} \\ x \equiv b \pmod{q} \end{cases}$$

Admet une solution unique modulo $n = pq$.

Généralisation Soit $(p_1, p_2, \dots, p_n)$ des nombres premiers entre eux alors l'application :

$$\begin{array}{ccc} \mathbb{Z}/(p_1 p_2 \dots p_n)\mathbb{Z} & \simeq & \mathbb{Z}/p_1\mathbb{Z} \times \mathbb{Z}/p_2\mathbb{Z} \dots \times \mathbb{Z}/p_n\mathbb{Z} \\ x \bmod (p_1 p_2 \dots p_n) & \mapsto & (x \bmod p_1, x \bmod p_2, \dots, x \bmod p_n) \end{array}$$

est un isomorphisme d'anneau.

Alors pour $(p_1, p_2, \dots, p_n)$ des nombres premiers entre eux, et $(a_1, a_2, \dots, a_n)$ des entiers on aura le système de congruence suivant :

$$\begin{cases} x \equiv a_1 \pmod{p_1} \\ x \equiv a_2 \pmod{p_2} \\ \vdots \\ x \equiv a_n \pmod{p_n} \end{cases} \quad (1.9)$$

Qui possède une unique solution unique modulo $n = p_1 \times p_2 \times \dots \times p_n$

Démonstration 1.2.2 (Correction RSA). Nous savons que $ed \equiv 1 \pmod{\varphi(n)}$.

Puisque $\varphi(n) = (p-1)(q-1)$, il existe un entier k tel que :

$$ed = 1 + k \times \varphi(n) \quad (1.10)$$

Donc on doit prouver que

$$m^{ed} = m^{1+k \times \varphi(n)} \equiv m \pmod{n} \quad (1.11)$$

Nous allons prouver cela en utilisant le Théorème des Restes Chinois, en montrant que la congruence est vraie modulo p et modulo q . Dès on peut distinguer deux cas

Cas 1 $\gcd(m, n) = 1$
 $\gcd(m, n) = 1$ alors $m^{\varphi(n)} \equiv 1 \pmod{n}$

$$m^{ed} = m^{1+k \times \varphi(n)} \quad (1.12)$$

$$m^{ed} = m \times 1 \pmod{n} = m \pmod{n} \quad (1.13)$$

donc la correction est bien vérifié pour le cas où $\gcd(m, n) = 1$

Cas 2 $\gcd(m, n) \neq 1$ donc m est multiple de q ou multiple de p . Supposons que m est multiple de p (Les calculs sont similaires dans le cas où m est multiple de q)

Vérification $\pmod{p}$

On a $m \equiv 0 \pmod{p}$

c'est à dire $m^{ed} \equiv 0 \pmod{p}$

donc l'égalité $m^{ed} = m$ est vérifié modulo p .

Vérification $\pmod{q}$

On a $\gcd(m, q) \neq 1$ donc :

$$m^{q-1} \equiv 1 \pmod{q} \quad (1.14)$$

comme $ed = 1 + k \times \varphi(n) = 1 + (q-1)(p-1)$ alors :

$$m^{ed} \equiv m^{1+k(p-1)(q-1)} \equiv m \cdot (m^{q-1})^{k(p-1)} \equiv m \cdot (1)^{k(p-1)} \equiv m \pmod{q} \quad (1.15)$$

Nous avons établi que $m^{ed} \equiv m \pmod{p}$ et $m^{ed} \equiv m \pmod{q}$. Puisque p et q sont premiers entre eux, le **Théorème des Restes Chinois** nous permet de conclure que cette congruence est vraie modulo leur produit $n = pq$:

$$m^{ed} \equiv m \pmod{n} \quad (1.16)$$

La correction du cryptosystème RSA est ainsi démontrée

$$(m^e)^d \equiv m \pmod{n}, \forall m \in \mathbb{Z}/n\mathbb{Z} \quad (1.17)$$

1.3 Optimisation par le théorème des restes chinois (CRT)

L'opération la plus coûteuse dans RSA est le déchiffrement, $m \equiv c^d \pmod{n}$. L'exposant d est généralement grand (de la même taille que n), rendant cette exponentiation modulaire lente.

Le détenteur de la clé privée connaît p et q . Il peut utiliser le CRT pour réduire la complexité de ce calcul, l'algorithme devient 4 fois plus rapide en utilisant le théorème des restes chinois.

Au lieu de calculer $m' \equiv c^d \pmod{n}$ en une seule fois, on calcule deux valeurs plus petites :

1. $m_p \equiv c^d \pmod{p}$
2. $m_q \equiv c^d \pmod{q}$

Ces calculs sont plus rapides car le module et l'exposant peuvent être réduits.

$$\begin{aligned} m_p &\equiv c^{d \pmod{p-1}} \pmod{p} \\ m_q &\equiv c^{d \pmod{q-1}} \pmod{q} \end{aligned}$$

On pré-calcule donc :

- $d_p = d \pmod{p-1}$
- $d_q = d \pmod{q-1}$

Les étapes du déchiffrement optimisé sont :

1. Calculer $m_p = c^{d_p} \pmod{p}$.
2. Calculer $m_q = c^{d_q} \pmod{q}$.
3. Calculer $h \equiv q^{-1}(m_p - m_q) \pmod{p}$.
4. Le message final est $m' = m_q + hq$.

Preuve 1.3.1. *Preuve que $m' = m$:*

Vérification $\pmod{q}$: Prenons l'équation de m' modulo q :

$$m' \equiv m_q + hq \equiv m_q q \equiv m \pmod{q}$$

Vérification $\pmod{p}$: Prenons l'équation de m' modulo p :

$$m' \equiv m_q + hq \pmod{p} \tag{1.18}$$

On a $h \equiv q^{-1}(m_p - m_q) \pmod{p}$, donc :

$$m' \equiv m_q + (q^{-1}(m_p - m_q))q \equiv m_q + m_p - m_q \equiv m_p \equiv m \pmod{p}$$

Puisque $m' \equiv m \pmod{p}$ et $m' \equiv m \pmod{q}$ et p et q sont premiers entre eux alors le théorème des restes chinois assure que :

$$m' \equiv m \pmod{pq}$$

$$m' \equiv m \pmod{n}$$

Donc m' correspond au message originale m .

Calcul de complexité

Analysons la complexité théorique en fonction de k , où k est le nombre de bits du module n .

Sans le théorème des restes chinois

Dans la méthode standard, le déchiffrement RSA consiste à calculer $m \equiv c^d \pmod n$. La complexité d'une exponentiation modulaire par la méthode "square and multiply" est en $O(\log(d) \times M(k))$, où $M(k)$ désigne le coût d'une multiplication sur k bits.

On suppose que d est de même taille que n ce qui est pratique pour renforcer la sécurité, alors $\log(d) \approx k$, ce qui nous donne $O(k \times M(k))$

Puisque $M(k)$ est le coût de la multiplication naïve de deux entiers de taille k alors $M(k) = O(k^2)$

Ceci nous donne une complexité de $O(k^3)$ pour une exponentiation modulo n

Avec le théorème des restes chinois

Lorsque l'on utilise le théorème des restes chinois (CRT), on remplace cette exponentiation par deux exponentiations modulo p et q , qui sont de taille $k/2$ bits :

$$m_p \equiv c^{d_p} \pmod p, \quad m_q \equiv c^{d_q} \pmod q.$$

Chaque exponentiation mod p ou mod q a alors un coût $O((k/2)^3) = O(k^3/8)$. Alors les deux exponentiations réunies coûtent environ :

$$O(k^3/8) + O(k^3/8) = 2 \times O(k^3/8) = O(k^3/4) = \frac{1}{4}O(k^3)$$

.

On en déduit que le déchiffrement RSA optimisé par le CRT est environ quatre fois plus rapide que le déchiffrement RSA standard.

Chapitre 2

Paramètres de sécurité de RSA

Le cryptosystème RSA permet de garantir l'authenticité du message mais sous certaines conditions. Tout le cryptosystème est basé sur la difficulté de factoriser le module n . Donc un mauvais choix des nombres premiers p et q ou de l'exposant e peut entraîner une vulnérabilité ce qui veut dire que le système cédera assez facilement aux attaques.

2.1 Choix des nombres premiers p et q

Les deux nombres premiers p et q sont à la base de la clé privée et leur produit n forme la base de la clé publique.

2.1.1 Taille de p et q

Pour assurer une bonne sécurité n doit être d'une taille minimale de 2048 bits.

Par conséquent, p et q doivent être des nombres premiers de grande taille, chacun ayant une longueur binaire d'environ la moitié de celle de n (1024 bits).

Supposons qu'un administrateur système choisisse des nombres premiers p et q qui sont très petits, même s'ils sont très éloignés l'un de l'autre.

Soient les paramètres suivants :

- $p = 7$
- $q = 131$

La clé publique contient le module $n = p \times q = 7 \times 131 = 917$.

L'Attaque (Factorisation par division) Un attaquant voit simplement $n = 917$. Il peut tenter une factorisation par *trial division* (division par essais). Cette méthode consiste à essayer de diviser n par tous les nombres premiers k jusqu'à $\sqrt{n}$.

1. **Calcul de la borne** : L'attaquant calcule la racine carrée de n :

$$\sqrt{n} = \sqrt{917} \approx 30.28$$

Il sait qu'il lui suffit de tester les nombres premiers ≤ 30 , soit : $\{2, 3, 5, 7, 11, 13, 17, 19, 23, 29\}$.

2. **Itérations de l'attaque** :

- $917 \div 2 = 458.5$ (Échec)
- $917 \div 3 \approx 305.66$ (Échec)
- $917 \div 5 = 183.4$ (Échec)
- $917 \div 7 = 131$ (**Succès !**)

3. **Factorisation** : L'attaquant a trouvé les facteurs en seulement 4 divisions :

$$p = 7 \quad \text{et} \quad q = 131$$

Conclusion Le système est cassé instantanément. La vulnérabilité fondamentale ici est la **taille de n** . Si n est petit, la borne $\sqrt{n}$ est également petite, rendant la liste des facteurs premiers à tester trivialement courte pour un ordinateur. C'est pourquoi un module n de 2048 bits est nécessaire : $\sqrt{2^{2048}} = 2^{1024}$, un nombre astronomiquement grand qui prend beaucoup de temps pour le calculer.

Prenons par exemple un ordinateur assez puissant capable d'effectuer 10^{12} opérations par seconde, et le nombre d'opérations nécessaire pour casser un système est de 2^{1024} . Donc le temps nécessaire pour effectuer ce calcul est de :

$$\begin{aligned} 2^{1024} &\approx 1,8 \times 10^{308} \\ \frac{2^{1024}}{10^{12}} &\approx \frac{1,8 \times 10^{308}}{10^{12}} = 1,8 \times 10^{296} \text{ secondes} \\ &\approx \frac{1,8 \times 10^{296}}{3,15 \times 10^7} \simeq 5,7 \times 10^{288} \text{ années} \end{aligned}$$

Donc le temps de calcul est infini et il n'y a aucun ordinateur capable de le calculer sachant que le calculateur le plus puissant du monde peut faire 10^{18} opérations par seconde.

2.1.2 Proximité

Un paramètre de sécurité fondamental est que p et q ne doivent pas être trop proches l'un de l'autre.

Justification (Attaque par factorisation de Fermat) : Si p et q sont proches, leur moyenne arithmétique $a = \frac{p+q}{2}$ est extrêmement proche de $\sqrt{n}$. Nous pouvons quantifier cette proximité précisément en fonction de la différence $|p - q|$.

Partons de l'identité remarquable :

$$(p + q)^2 - (p - q)^2 = 4pq = 4n$$

En isolant la somme $p + q$, nous obtenons :

$$p + q = \sqrt{4n + (p - q)^2} = 2\sqrt{n + \frac{(p - q)^2}{4}}$$

En factorisant par $2\sqrt{n}$, on met en évidence l'écart :

$$p + q = 2\sqrt{n} \cdot \sqrt{1 + \left(\frac{p - q}{2\sqrt{n}}\right)^2}$$

Cette équation montre que $p + q \approx 2\sqrt{n}$ si et seulement si le terme $\frac{p-q}{2\sqrt{n}}$ est très petit.

Dans l'attaque de Fermat, on pose $a = \lceil \sqrt{n} \rceil$. On cherche alors b tel que $n = a^2 - b^2$, c'est-à-dire $b^2 = a^2 - n$. Si la différence $|p - q|$ est petite, alors a est très proche de $\sqrt{n}$, et $b = \frac{|p-q|}{2}$ est un petit entier, ce qui permet de trouver b (et donc la factorisation) très rapidement en testant des valeurs de a incrémentales à partir de $\lceil \sqrt{n} \rceil$.

2.2 Choix de l'exposant public

L'exposant public e est utilisé pour le chiffrement : $c \equiv m^e \pmod{n}$. Il doit satisfaire deux conditions : $1 < e < \varphi(n)$ et $\text{pgcd}(e, \varphi(n)) = 1$. L'opération de chiffrement doit être aussi rapide que possible.

2.2.1 Danger de l'exposant publique trop petit

Choisir un exposant e trop petit optimise le temps de chiffrement et déchiffrement mais peut entraîner différents types d'attaques et rendre le système assez vulnérable. Il existe plusieurs types d'attaques que le petit exposant peut entraîner

Attaque sur les petits messages

Lorsque le message m est trop petit son chiffrement $m^e < n$ donc le mod (n) n'as plus d'effet. Donc l'attaquant peut retrouver le message m en calculant simplement la racine e -ième de notre texte chiffré c dans $\mathbb{R}$

$$m = \sqrt[e]{c}$$

Cette opération est instantanée pour un ordinateur contrairement à la racine modulaire. Dans ce cas RSA qui est un problème difficile est transformé en un problème facile, ce qui rend le système vulnérable.

Exemple : Supposons qu'un administrateur génère des clés RSA avec un module n modeste et un exposant public très petit pour accélérer les calculs.

- **Paramètres :** $p = 101$ et $q = 113$. Le module est $n = 11413$.
- **Clé publique :** $(n = 11413, e = 3)$.
- **Message :** On souhaite chiffrer le nombre $m = 20$.

Le Chiffrement :

L'expéditeur calcule le chiffré c avec la formule standard :

$$c \equiv m^e \pmod{n}$$

$$c \equiv 20^3 \pmod{11413}$$

$$c \equiv 8000 \pmod{11413}$$

Comme $8000 < 11413$, le modulo n'a aucun effet réducteur. Le message chiffré envoyé est simplement $c = 8000$.

L'Attaque :

L'attaquant intercepte $c = 8000$. Il connaît la clé publique $e = 3$. Il suspecte que le message original était petit. Au lieu de chercher à factoriser n , il calcule simplement la racine cubique réelle de c :

$$m = \sqrt[3]{8000} = 20$$

Le message est retrouvé instantanément sans la clé privée d .

Attaque par diffusion de Hastad

L'attaque par diffusion de Hastad aussi appelé en anglais **Broadcast attack** est une attaque redoutable si le même message est envoyé à au moins k destinataires utilisant le même petit exposant e tel que $k \geq e$. Cette attaque est basée sur le théorème des restes chinois.

Soit m le message clair qui est chiffré en utilisant e pour être envoyé à plusieurs destinataires utilisant un différent module

Soit $k \geq e$

$$\left\{ \begin{array}{l} c_1 \equiv m^e \pmod{n_1} \\ c_2 \equiv m^e \pmod{n_2} \\ . \\ . \\ c_e \equiv m^e \pmod{n_k} \end{array} \right. \quad (2.1)$$

Donc si notre attaquant intercepte les messages chiffrés $c_1, c_2, \dots, c_k$ et ils connaît les clés publiques $(n_1, e), (n_2, e), \dots, (n_k, e)$, et si les $n_i, i \in 1 \dots k$ sont premiers entre eux. Alors l'attaquant peut utiliser le théorème des restes chinois pour résoudre le système.

$$C \equiv m^e \pmod{n_1 \times n_2 \times \dots \times n_k} \quad (2.2)$$

Il est très probable que $m^e \leq n_1 \times n_2 \times \dots \times n_k$, ce qui nous ramène à la première attaque par factorisation l'attaquant devra juste calculer : $\sqrt[k]{C} \equiv m$ et il a réussi à décrypter le message.

2.2.2 Recommandation pour le choix de l'exposant public e

Pour un système sécurisé et performant La valeur standard pour e actuellement est $e = 65537 = 2^{16} + 1$

Le choix de cette valeur se repose sur deux raisons principale :

La sécurité : Une valeur suffisamment grande pour éviter les attaques sur les petits exposants.

La performance : La valeur $65537 = 2^{16} + 1$ il s'écrit donc En binaire 100000000000000001. Il ne contient que deux bits à '1'. L'algorithme d'exponentiation modulaire (Square-and-Multiply) utilisé pour le chiffrement, dépend du nombre de bits à '1' le calcul est extrêmement rapide pour $e = 65537$.

L'utilisation de $e = 65537$ est aujourd'hui la norme recommandée . Si e est choisi aléatoirement et est très grand, le chiffrement devient inutilement lent sans gain de sécurité significatif.

2.2.3 Risques liées à un exposant d très petit

Tout comme pour l'exposant public e , il peut être tentant de choisir un exposant privé d petit dans le but d'optimiser cette fois-ci le temps de déchiffrement. Cette optimisation est particulièrement recherchée pour les appareils à faible puissance de calcul, comme les cartes à puces, où la génération de signature ou le déchiffrement doit être rapide.

Cependant, choisir un d petit est beaucoup plus dangereux que de choisir un e petit. Si l'exposant privé d est trop petit par rapport au module n , la complexité du problème s'effondre. On a :

$$ed - k\varphi(n) = 1 \quad (2.3)$$

Pour un d petit, cette équation permet à un attaquant de retrouver d en utilisant uniquement la clé publique (n, e) .

Il a été démontré que si la valeur de d est inférieure à environ un tiers de la racine quatrième de n :

$$d < \frac{1}{3}n^{\frac{1}{4}}$$

Alors le système est totalement insécurisé. L'attaquant peut retrouver la clé privée efficacement en utilisant le développement en fractions continues de $\frac{e}{n}$. Cette vulnérabilité spécifique est exploitée par l'attaque de Wiener, que nous détaillerons et analyserons en profondeur dans le chapitre suivant.

Chapitre 3

Attaque de Wiener

Pour accélérer le chiffrement ou le déchiffrement du protocole RSA, il peut être tentant de choisir un exposant public ou secret plus petit que le module n . Ce choix est très intéressant lorsque on utilise des machines avec des capacités de calculs limitées comme les cartes à puces. Mais comme vu dans le chapitre précédant cela peut conduire vers de diverses vulnérabilités dans le système. En 1990, Wiener a exhibé une attaque que peut entraîner un exposant secret trop petit d , il a montré qu'on peut le retrouver à partir de la clé publique (n, e) si sa taille ne dépasse pas le quart de la taille du module n et p et q sont de la même taille.

Wiener a montré que si $d < n^{\frac{1}{4}}$, alors d est le dénominateur d'une certaine convergente de la fraction continue de $\frac{e}{n}$. Donc d peut être calculé à partir de la clé publique (n, e) . Ce résultat est basé sur le théorème de Legendre sur les fractions continues.

3.1 Les fractions continues

Théorème 3.1.1. *Tout nombre réel positif x a une écriture unique comme fraction continue :*

$$x = [q_0, q_1, q_2, \dots, q_m] = q_0 + \frac{1}{q_1 + \frac{1}{q_2 + \frac{1}{\dots + \frac{1}{q_m}}}}$$

Où les q_i sont des entiers positifs, De plus, la fraction continue est finie si et seulement si le nombre x est rationnel.

Définition 3.1.1. Soit $[q_0, q_1, q_2, \dots]$ une fraction continue d'un nombre x ,

1. Les nombres entiers q_i s'appellent les quotients partiels.
2. Les nombres rationnels $[q_0, q_1, q_2, \dots, q_k]$ s'appellent les convergentes

(réduites) de x .

$$[q_0, q_1, q_2, \dots, q_k] = q_0 + \frac{1}{q_1 + \frac{1}{q_2 + \frac{1}{\dots + \frac{1}{q_k}}}}$$

Les convergentes forment une suite de nombres rationnels qui constituent les meilleures approximations rationnelles de x .

3.1.1 Le développement en fraction continue

Le développement en fraction continue d'un nombre rationnel positif x est construit en retranchant la partie entière de x , puis en inversant à chaque fois le reste et en retranchant à nouveau la partie entière, jusqu'à ce que le reste soit nul.

On note q_i le quotient entier et r_i le reste à l'étape i , et l'on note m le nombre d'étapes d'inversion, avec :

$$\begin{aligned} q_0 &= \lfloor x \rfloor, & r_0 &= x - q_0, & \text{et} \\ q_i &= \lfloor \frac{1}{r_{i-1}} \rfloor, & r_i &= \frac{1}{r_{i-1}} - q_i, & \text{pour } 1 \leq i \leq m \end{aligned}$$

3.1.2 Calcul des convergentes

À chaque étape i du développement, on peut former une fraction $\frac{k_i}{d_i}$ appelée i -ème convergente.

Les convergentes se calculent par récurrence sans avoir besoin de recalculer toute la fraction à chaque fois :

— **Initialisation :**

$$\begin{aligned} k_0 &= q_0, & d_0 &= 1 \\ k_1 &= q_1 q_0 + 1, & d_1 &= q_1 \end{aligned}$$

— **Récurrence (pour $i \geq 2$) :**

$$\begin{aligned} k_i &= q_i k_{i-1} + k_{i-2} \\ d_i &= q_i d_{i-1} + d_{i-2} \end{aligned}$$

Théorème 3.1.2. Théorème de Legendre

Soit α un nombre réel et $\frac{p}{q}$ un nombre rationnel avec $q > 0$ et p et q premiers entre eux.

Si

$$\left| \alpha - \frac{p}{q} \right| < \frac{1}{2q^2}$$

Alors $\frac{p}{q}$ est nécessairement une convergente de la fraction continue de α .

3.2 Principe mathématique de l'attaque de Wiener

Wiener a décrit un algorithme qui dans un temps polynomiale peut casser un système RSA, si les deux conditions ci dessous sont vérifiées :

- $d < \frac{1}{3}n^{0.25}$, la taille maximale de d est d'un quart de la taille de n .
- $q < p < 2q$, p et q sont de même taille.

Alors, nous allons démontrer que l'écart entre $\frac{e}{n}$ et $\frac{k}{d}$ vérifie l'inégalité $\left| \frac{e}{n} - \frac{k}{d} \right| < \frac{1}{2d^2}$, ce qui est la condition requise par le théorème de Legendre.

Démonstration 3.2.1. *Partons de la relation des exposants.*

$$ed \equiv 1 \pmod{\phi(n)} \quad (3.1)$$

Alors il existe un entier k tel que :

$$ed - k\phi(n) = 1 \quad (3.2)$$

Puisque $ed > 1$ donc k est forcément un entier positif. L'attaquant connaît n et e , mais ne connaît pas $\phi(n)$. Cependant, pour de grandes valeurs de p et q , la valeur de $\phi(n)$ est très proche de n . En effet :

$$\phi(n) = (p-1)(q-1) = n - p - q + 1$$

Comme p et q sont de même taille (environ $\sqrt{n}$), on peut estimer que $p + q \approx 2\sqrt{n}$. Plus précisément, pour être large et assurer la précision l'inégalité suivante a été choisie

$$n - 3\sqrt{n} < \phi(n) < n$$

Cela implique que $n - \phi(n) < 3\sqrt{n}$.

Reprenons l'équation $ed - k\phi(n) = 1$. Divisons par nd :

$$\begin{aligned} \frac{ed - k\phi(n)}{nd} &= \frac{1}{nd} \\ \frac{e}{n} - \frac{k\phi(n)}{nd} &= \frac{1}{nd} \\ \frac{e}{n} - \frac{k(n - (n - \phi(n)))}{nd} &= \frac{1}{nd} \\ \frac{e}{n} - \frac{kn}{nd} + \frac{k(n - \phi(n))}{nd} &= \frac{1}{nd} \\ \frac{e}{n} - \frac{k}{d} &= \frac{1}{nd} - \frac{k(n - \phi(n))}{nd} \end{aligned}$$

On applique la valeur absolue et on obtient :

$$\left| \frac{e}{n} - \frac{k}{d} \right| = \left| \frac{1 - k(n - \phi(n))}{nd} \right| \approx \frac{k(n - \phi(n))}{nd}$$

En utilisant l'inégalité $n - \varphi(n) < 3\sqrt{n}$, nous avons :

$$\left| \frac{e}{n} - \frac{k}{d} \right| < \frac{k3\sqrt{n}}{nd} = \frac{3k}{d\sqrt{n}}$$

Et comme $d < \frac{1}{3}n^{0.25}$ a été choisie spécifiquement pour satisfaire la condition du théorème de Legendre. Alors ceci entraîne que :

$$\frac{3k}{d\sqrt{n}} < \frac{1}{2d^2}$$

Ce qui nous entraîne vers l'inégalité ci dessous :

$$\left| \frac{k}{d} - \frac{e}{n} \right| < \frac{1}{2d^2}$$

Puisque $\left| \frac{e}{n} - \frac{k}{d} \right| < \frac{1}{2d^2}$, nous pouvons conclure par le théorème de Legendre que $\frac{k}{d}$ est une convergente du développement en fraction continue de $\frac{e}{n}$. Cela permet à l'attaquant de retrouver notre exposant secret d en calculant simplement les fractions continues de $\frac{e}{n}$, qui sont toutes les deux des valeurs publiques.

3.3 Conditions de réussite de l'attaque

L'attaque de Wiener repose sur la capacité à utiliser le théorème de Legendre. Ce théorème garantit que $\frac{k}{d}$ est une réduite de $\frac{e}{n}$ si l'erreur d'approximation respecte l'inégalité stricte :

$$\left| \frac{e}{n} - \frac{k}{d} \right| < \frac{1}{2d^2} \quad (3.3)$$

D'autre part, nous avons la majoration suivante en fonction de la taille de n :

$$\left| \frac{e}{n} - \frac{k}{d} \right| < \frac{3k}{d\sqrt{n}} \quad (3.4)$$

Pour que l'attaque réussisse, nous cherchons à pouvoir appliquer le théorème de Legendre donc à satisfaire l'inégalité suivante :

$$\frac{3k}{d\sqrt{n}} < \frac{1}{2d^2}$$

Afin de satisfaire cette dernière on cherche à trouver la taille de la clé privée qui assurera que l'attaque réussisse.

$$\frac{3k*2d^2}{d} < \sqrt{n}$$

$$6kd < \sqrt{n}$$

Nous avons $ed - k\varphi(n) = 1$ ce qui entraîne $\frac{k}{d} \approx \frac{e}{\varphi(n)}$. Nous savons que dans le système RSA $e < n$ et pour de grandes valeurs n est donc proche de $\varphi(n)$ donc $e < \varphi(n)$ donc $k < d$.

$$6kd < 6d^2 < \sqrt{n}$$

$$d^2 < \frac{1}{6}\sqrt{n} \implies d < \frac{1}{\sqrt{6}}n^{1/4}$$

Comme $\sqrt{6} \approx 2.45$, Wiener a établi une borne légèrement plus conservative pour assurer la réussite de l'attaque. Il a ainsi fixé la condition finale à :

$$d < \frac{1}{3}n^{\frac{1}{4}}$$

Ce résultat démontre que si la clé privée d possède un nombre de bits inférieur au quart du nombre de bits du module n , le système RSA est vulnérable.

Exemple 3.3.1. Pour une clé RSA standard de 1024 bits ($n \approx 2^{1024}$), l'attaque de Wiener permet de retrouver instantanément toute clé privée d d'une taille inférieure à environ 256 bits ($n^{0.25} = 2^{256}$).

3.4 Algorithme de Wiener

Nous avons établi précédemment que si $d < \frac{1}{3}n^{0.25}$ et p et q sont de même taille, alors $\frac{k}{d}$ est une convergente du développement en fraction continue de $\frac{e}{n}$. Dans cette partie nous détaillerons les étapes à suivre pour générer ces convergentes et vérifier laquelle correspond à la clé privée.

Vous retrouverez une implémentation de l'algorithme de Wiener à l'annexe.

3.4.1 Développement en fraction continue

On a $\frac{e}{n}$ est un nombre rationnel il peut donc s'écrire de manière unique sous la forme d'une fraction continue finie :

$$\frac{e}{n} = [q_0; q_1, q_2, \dots, q_m] = q_0 + \frac{1}{q_1 + \frac{1}{q_2 + \frac{1}{\dots + \frac{1}{q_m}}}}$$

On note q_i le quotient entier et r_i le reste à l'étape i , et l'on note m le nombre d'étapes d'inversion, avec :

$$\begin{aligned} q_0 &= \lfloor x \rfloor, & r_0 &= x - q_0, & \text{et} \\ q_i &= \lfloor \frac{1}{r_{i-1}} \rfloor, & r_i &= \frac{1}{r_{i-1}} - q_i, & \text{pour } 1 \leq i \leq m \end{aligned}$$

3.4.2 Calcul des convergentes

À chaque étape i du développement, on peut former une fraction $\frac{k_i}{d_i}$ appelée i -ème convergente, où k_i est une approximation du coefficient k de l'équation d'Euler et d_i un potentiel candidat pour l'exposant privé d que nous voulons retrouver. Cette fraction est la meilleure approximation rationnelle de $\frac{e}{n}$ pour un dénominateur donné.

Les convergentes se calculent par récurrence sans avoir besoin de recalculer toute la fraction à chaque fois :

— **Initialisation :**

$$\begin{aligned} k_0 &= q_0, & d_0 &= 1 \\ k_1 &= q_1 q_0 + 1, & d_1 &= q_1 \end{aligned}$$

— **Récurrence (pour $i \geq 2$) :**

$$\begin{aligned} k_i &= q_i k_{i-1} + k_{i-2} \\ d_i &= q_i d_{i-1} + d_{i-2} \end{aligned}$$

Ainsi, l'ensemble des candidats potentiels pour la clé privée d est l'ensemble des dénominateurs $\{d_0, d_1, \dots, d_m\}$ générés par cette suite.

3.4.3 Vérification des candidats

Pour tout couple (k_i, d_i) , nous devons vérifier s'il permet de factoriser n . L'algorithme de vérification se déroule comme suit :

1. **Vérification de la relation $ed_i - k_i\varphi(n) = 1$:** On a déjà montré que $ed - k\varphi(n) = 1$, Donc si d_i est la bonne clé secrète, alors le couple (k_i, d_i) doit vérifier que $ed_i - k_i\varphi(n) = 1$. Sinon d_i est rejeté
2. **Calcul de d'une potentielle φ' :** On calcule une valeur potentielle pour l'indicatrice d'Euler :

$$\varphi' = \frac{ed_i - 1}{k_i}$$

Si φ' n'est pas un entier, d_i est rejeté.

3. **Résolution de l'équation quadratique :** Nous savons que $\varphi(n) = (p-1)(q-1) = n - (p+q) + 1$, ce qui nous donne l'égalité suivante :

$$p + q = n - \varphi' + 1$$

Les facteurs premiers p et q sont donc les racines du polynôme :

$$X^2 - (n - \varphi' + 1)X + n = 0$$

4. **Test du discriminant** : On calcule le discriminant $\Delta = (n - \varphi' + 1)^2 - 4n$. Pour que p et q soient des entiers, Δ doit être un carré parfait. Si $\sqrt{\Delta}$ est un entier, alors nous avons trouvé la factorisation de n :

$$p = \frac{(n - \varphi' + 1) + \sqrt{\Delta}}{2}, \quad q = \frac{(n - \varphi' + 1) - \sqrt{\Delta}}{2}$$

Si $pq = n$, l'attaque est un succès et nous avons trouvé notre clé secrète $d = d_i$.

Chapitre 4

Le standard de chiffrement PKCS#1 v1.5

Jusqu'à présent, nous avons étudié le RSA sous sa forme scolaire (Text-book RSA), qui est la forme algébrique brute du RSA. Dans ce modèle, le chiffrement d'un message m se résume à l'opération $c \equiv m^e \pmod{n}$. Bien que mathématiquement correcte, cette approche naïve est inutilisable en pratique pour des raisons de sécurité majeures.

Le RSA scolaire est déterministe : chiffrer deux fois le même message avec la même clé publique produira toujours le même cryptogramme c . Un attaquant qui observe le trafic peut effectuer une attaque par dictionnaire : s'il soupçonne que le message est "OUI" ou "NON", il lui suffit de chiffrer ces deux mots avec la clé publique et de comparer le résultat avec le message intercepté. Pour garantir la sécurité, le chiffrement doit être probabiliste : deux chiffrements du même message doivent produire des résultats différents. Il y a aussi le problème des petits messages, si m est très court et l'exposant e petit, il se peut que $m^e < n$. Dans ce cas, l'opération modulo n ne s'applique pas, et le message peut être retrouvé par une simple racine e -ième dans les entiers naturels.

Pour contrer ces problèmes, On doit utiliser un schéma de remplissage (Padding) avant de chiffrer. Ce procédé permet d'ajouter de l'aléa et de structurer le message. C'est précisément le rôle du standard PKCS#1 v1.5 (Public-Key Cryptography Standards).

Définition 4.0.1. Le système hexadécimal

L'hexadécimal est un système de représentation des caractères en base 16 il utilise les chiffres de 0 à 9 et les lettres de A à F (où A vaut 10, et F vaut 15). Il est massivement utilisé car il permet de représenter le système binaire de façon beaucoup plus compacte et lisible pour l'œil humain. Un caractère hexadécimal correspond exactement à 4 bits. Par conséquent, un octet complet se représente toujours par exactement deux caractères hexadécimaux (par exemple, 00 pour 0, et FF pour 255).

Exemple 4.0.1. Prenons l'exemple du mot « cryptologie », la conversion s'effectue lettre par lettre :

- **c** → 63
- **r** → 72
- **y** → 79
- **p** → 70
- **t** → 74
- **o** → 6F
- **l** → 6C
- **o** → 6F
- **g** → 67
- **i** → 69
- **e** → 65

Ainsi, la chaîne de caractères « cryptologie » sera représentée par la suite hexadécimale suivante :

63 72 79 70 74 6F 6C 6F 67 69 65.

4.1 Le format PKCS1 v1.5

Le chiffrement d'un message m en RSA ne se fait pas directement. Il est nécessaire de suivre un standard précis pour le chiffrer d'une façon plus sécurisé.

4.1.1 Le chiffrement

Le processus de chiffrement se sépare en quatre étapes :

1. **Le formatage du bloc de chiffrement** : Cette étape consiste à construire le bloc structuré EB (Encryption Block) à partir du message original M . Le bloc EB doit être de même longueur que le module n
2. **La conversion de la chaîne d'octets en entier** : Le bloc formaté EB est donc converti en un grand nombre entier x .
3. **Le calcul RSA** :

$$y = x^e \mod n$$

4. **La conversion de l'entier en chaîne d'octets** : y est reconverti en une suite d'octets de longueur fixe (égale à la longueur de n) pour former le cryptogramme final c .

Formatage du bloc EB

Selon le standard PKCS#1 v1.5, le bloc de chiffrement EB (Encryption Block) doit avoir une longueur k égale à la longueur du module n en octets. Il est construit par la concaténation suivante :

$$EB = 00 \parallel BT \parallel PS \parallel 00 \parallel M$$

Où les composants sont définis comme suit :

- **00** : Un octet nul initial. Il garantit que l'entier représenté par EB est strictement inférieur au module n .
- **BT** : L'indicateur de type de bloc (Block Type). Ce bloc indique le type du chiffrement. Pour un chiffrement à clé secrète la valeur du **BT** sera 00 ou 01 ; la valeur 01 est la norme recommandée car elle impose un remplissage de FF ce qui maximise la valeur numérique du bloc, tandis que la valeur 00 est rarement utilisée car elle impose un remplissage de zéros et exige que le message ne commence pas par un zéro. Quant au chiffrement à clé publique **BT** sera 02.
- **PS (Padding String)** : Une chaîne d'octets de remplissage. elle est de longueur $k - 3 - \text{len}(M)$ octets.
 - si le **BT** est 00 alors les octets du **PS** seront 00.
 - si le **BT** est 01 alors les octets du **PS** seront FF.
 - si le **BT** est 02 alors les octets du **PS** seront générés d'une façon pseudo-aléatoire (Généré par un algorithme mathématiques produisant des valeurs au hasard pour s'assurer que deux blocs identiques ne donnent pas le même chiffré) et différent de zéro.
- **00** : Un octet séparateur. Il marque sans ambiguïté la fin du padding et le début du message utile.
- **M** : Le message clair à chiffrer.

Si la longueur du message est trop grande par rapport à la taille du module, le formatage est impossible. La longueur du padding PS est calculée par :

$$\text{len}(PS) = k - \text{len}(M) - 3$$

Ce qui impose que la taille du message M soit inférieure ou égale à $k - 11$ octets, sachant que la longueur du PS est au moins de 8 octets.

La conversion de la chaîne d'octets en entier

Le bloc de chiffrement **EB** doit être converti en un entier x . Soit $(EB_1, EB_2, \dots, EB_k)$ Les octets bloc de chiffrement, alors :

$$x = \sum_{i=1}^k 2^{8(k-i)} \times EB_i$$

Le calcul RSA

Le bloc de chiffrement entier x doit être élevé à la puissance e modulo n pour donner un entier y , qui correspond aux données chiffrées.

$$y = x^e \pmod{n}$$

La conversion de l'entier en chaîne d'octets

L'entier représentant les données chiffrées, noté y , doit être converti en une chaîne d'octets ED (Encrypted Data) de longueur k .

Les octets $(ED_1, ED_2, \dots, ED_k)$ sont déterminés de manière à satisfaire l'équation suivante :

$$y = \sum_{i=1}^k 2^{8(k-i)} \times ED_i$$

4.1.2 Le déchiffrement

Tout comme le chiffrement le déchiffrement aussi doit suivre un standard précis pour déchiffrer. Le processus du déchiffrement se sépare en quatre étapes :

1. **La conversion de la chaîne d'octets en entier**
2. **Le calcul RSA :**
3. **Conversion de l'entier en chaîne d'octets :**
4. **Analyse du bloc de chiffrement**

La conversion de la chaîne d'octets en entier

La chaîne d'octets reçu ED (Encrypted Data) est convertie en un entier y en utilisant l'équation suivante :

$$y = \sum_{i=1}^k 2^{8(k-i)} \times ED_i$$

Le calcul RSA

Les données chiffrées entières y doivent être élevées à la puissance d modulo n pour donner un entier x (ou d est l'exposant secret), correspondant au bloc de chiffrement entier.

$$x = y^d \pmod{n}$$

Conversion de l'entier en chaîne d'octets

Le bloc de chiffrement entier x doit être converti en une chaîne d'octets EB de longueur k , correspondant au bloc de chiffrement, cela en suivant l'équation suivante :

$$x = \sum_{i=1}^k 2^{8(k-i)} \times EB_i$$

Analyse du bloc de chiffrement

Cette étape consiste à décomposer (parser) le bloc de chiffrement EB obtenu à l'étape précédente pour en extraire le type de bloc BT , la chaîne de remplissage PS et enfin les données M (ou D).

Le récepteur vérifie la conformité de la structure :

$$EB = 00 \parallel BT \parallel PS \parallel 00 \parallel M$$

Selon le standard, une erreur doit être levée si l'une des conditions suivantes se produit :

- **Type de bloc invalide** : Pour une opération de déchiffrement (clé privée), le type de bloc BT doit impérativement être égal à 02.
- **Structure ambiguë** : Le bloc EB doit contenir un octet séparateur 00 pour distinguer la fin du padding PS et le début du message M . Si aucun octet 00 n'est trouvé après les en-têtes, l'analyse échoue.
- **Padding trop court** : La chaîne de remplissage PS doit contenir au moins 8 octets.
- **Incohérence du padding** : Puisque $BT = 02$, le standard impose que le padding PS ne contienne que des octets non nuls. La présence d'un octet 00 dans le padding rendrait la structure ambiguë.

Si le bloc respecte toutes ces conditions, le message M est extrait et renvoyé. Sinon, une erreur de déchiffrement est signalée.

Exemple 4.1.1.

Prenons comme message clair M « Hello ». En utilisant la table ASCII standard, sa représentation hexadécimale est : **48 65 6C 6C 6F**.

Génération des clés RSA

- **Deux nombres premiers** :
 - $p = 14234567890123456789$
 - $q = 11394857612345678903$
- **Clé publique (n, e)** :
 - $n = p \times q = 162201538654298132578500123547896321457$

(Le module n est composé de 39 chiffres, ce qui représente près 16 octets en mémoire).

— $e = 65537$

— **Clé privée (d) :**

$$d = e^{-1} \pmod{\varphi(n)} = 83921345678901234567890123456789012345$$

Supposons donc que nous communiquions avec ce module RSA n d'une taille $k = 16$ octets. Si nous reprenons notre message clair M (« Hello ») qui mesure 5 octets, la taille de la chaîne de remplissage PS doit être de :

$$\text{len}(PS) = k - 3 - \text{len}(M) = 16 - 3 - 5 = 8 \text{ octets.}$$

1. Le Chiffrement

1. **Formatage du bloc de chiffrement** : L'algorithme génère 8 octets pseudo-aléatoires non nuls pour le PS (par exemple : A3 5F 91 77 2B 88 4C D9).

Le bloc EB est assemblé :

00 02 A3 5F 91 77 2B 88 4C D9 00 48 65 6C 6C 6F

Voici la décomposition exacte de ce bloc :

- 00 : L'octet nul initial.
 - 02 : L'indicateur de type de bloc (BT) signifiant un chiffrement à clé publique.
 - A3 5F 91 77 2B 88 4C D9 : La chaîne de remplissage (PS). Ici, nous avons 8 octets non nuls générés de manière pseudo-aléatoire.
 - 00 : L'octet séparateur marquant la fin du remplissage.
 - 48 65 6C 6C 6F : Le message clair M (« Hello »).
2. **Conversion de la chaîne d'octets en entier** : L'algorithme interprète cette suite d'octets et la convertit en un grand entier naturel x :

$$x = 13345678901234567890123456789012345$$

3. **Le calcul RSA** : On utilise la clé publique (n, e) pour calculer $y = x^e \pmod{n}$. L'opération mathématique donne un nouvel entier :

$$y = 8520153865429813257850012354789632145$$

4. **Conversion de l'entier en chaîne d'octets** : L'entier y est reconverti en une chaîne d'octets ED de longueur $k = 16$. C'est le cryptogramme envoyé sur le réseau :

ED = 05 F2 B9 A1 C8 D7 E6 F4 A3 B2 C1 D0 E9 F8 A7 B6

2. Le Déchiffrement

1. **Conversion de la chaîne d'octets en entier** : Le destinataire reçoit les 16 octets ED et les convertit pour retrouver le grand entier décimal y :

$$y = 8520153865429813257850012354789632145$$

2. **Le calcul RSA** : Il utilise sa clé privée d pour calculer $x = y^d \pmod{n}$. Il retrouve l'entier x initial :

$$x = 13345678901234567890123456789012345$$

3. **Conversion de l'entier en chaîne d'octets** : L'entier x est retranscrit sous forme de 16 octets, ce qui restitue parfaitement le bloc EB formaté :

00 02 A3 5F 91 77 2B 88 4C D9 00 48 65 6C 6C 6F

4. **Analyse du bloc de chiffrement** : Le système parse le bloc :
 - Il valide la présence du 00 initial.
 - Il valide le type de bloc 02 (chiffrement à clé publique).
 - Il passe le padding jusqu'au séparateur 00.
 - Il extrait les données restantes M (48 65 6C 6C 6F), qui se traduisent par le message original : « Hello ».

Remarque 4.1.1. Limitation du standard

La limitation de la taille des messages à $K - 11$ octets soulève une légitime question quant à l'utilisation du standard PKCS#1 v1.5 en conditions réelles.

En réalité, ce standard n'est pas conçu pour chiffrer de longs messages, car c'est une opération trop coûteuse. Pour contourner cette limite les cryptographes optent plutôt pour un chiffrement hybride.

L'entièreté du message est chiffré par un chiffrement symétrique plus rapide ; L'AES par exemple ; en utilisant une clé secrète à usage unique appelé clé de session. C'est uniquement cette clé de session qui est formatée par le standard PKCS#1 v1.5 puis chiffré par RSA. La taille de cette clé est généralement entre 16 et 32 octets donc elle respecte largement les limitations du standard. Donc, l'attaque de Bleichenbacher ne vise pas à attaquer toutes les données transmises, mais cible spécifiquement le bloc RSA pour en extraire cette clé de session. Une fois cette clé compromise l'attaquant peut déchiffrer toutes les données instantanément.

4.2 L'Oracle de conformité PKCS#1 v1.5

Le fait de signaler une erreur lors du déchiffrement (comme vu à l'étape précédente) semble inoffensif. Cependant, c'est précisément ce message d'er-

reur qui introduit une vulnérabilité fatale, exploitée par Daniel Bleichenbacher en 1998. Cette vulnérabilité repose sur la notion d'« oracle de conformité ».

4.2.1 Définition de l'Oracle

En cryptographie, un oracle est modélisé comme une fonction en boîte noire à laquelle un attaquant a accès. Dans notre contexte, l'oracle de conformité représente le serveur ou le récepteur légitime qui possède la clé privée d . Lorsqu'on soumet un cryptogramme c à cet oracle, celui-ci effectue les opérations suivantes :

1. Il calcule $m = c^d \pmod{n}$.
2. Il analyse la structure de m .
3. Il renvoie une réponse binaire (1 bit d'information) :
 - **Vrai (1)** : Si le bloc est conforme au format PKCS#1 v1.5 (il commence par 00 02 et respecte les règles de padding).
 - **Faux (0)** : Si le format est invalide.

C'est cet oracle qui est le maillon faible : il transforme une simple erreur de formatage en une fuite d'information sur le message clair.

4.2.2 Modélisation Mathématique

Pour exploiter l'oracle de conformité, nous devons traduire la condition « le bloc commence par 00 02 » sous la forme d'une inégalité mathématique.

Soit k la taille du module n en octets. Rappelons que le bloc de chiffrement EB est converti en un entier m selon l'équation vue précédemment :

$$m = \sum_{i=1}^k 2^{8(k-i)} \times EB_i$$

Si le bloc est conforme au standard PKCS#1 v1.5, nous connaissons avec certitude la valeur de ses deux premiers octets :

- $EB_1 = 00$
- $EB_2 = 02$

Ce qui nous donne :

$$\begin{aligned} m &= (00 \times 2^{8(k-1)}) + (02 \times 2^{8(k-2)}) + \sum_{i=3}^k 2^{8(k-i)} \times EB_i \\ m &= 2 \times 2^{8(k-2)} + \sum_{i=3}^k 2^{8(k-i)} \times EB_i \end{aligned}$$

Posons $B = 2^{8(k-2)}$. L'équation de notre message devient alors :

$$m = 2B + R$$

Où R représente la valeur entière des octets restants (le padding et le message clair).

Pour déterminer l'intervalle dans lequel se trouve m , nous devons évaluer les bornes de ce reste R :

- **Borne inférieure** : La valeur minimale de R est atteinte si tous les octets suivants valent 00. Dans ce cas, $R = 0$. La valeur minimale de m est donc $2B + 0 = 2B$.
- **Borne supérieure** : La valeur maximale de R est atteinte si tous les octets restants valent FF (soit 255).

On a :

$$\sum_{i=3}^k 2^{8(k-i)} \times EB_i < B = 2^{8(k-2)}$$

Par conséquent, si l'on additionne $2B$ à cette dernière inégalité, on obtient $m < 3B$.

En conclusion, l'oracle répond **Vrai** si et seulement si l'entier m est borné par :

$$2B \leq m < 3B$$

Dans tous les autres cas, l'oracle répond **Faux**. C'est sur cette inégalité que se reposera toute la phase de réduction d'intervalles de l'attaque de Bleichenbacher.

Chapitre 5

Protocole d'attaque de Bleichenbacher

Dans le chapitre précédent, nous avons détaillé le standard de chiffrement PKCS#1 v1.5 et mis en évidence une vulnérabilité d'implémentation majeure : la présence d'un oracle de conformité. Nous avons modélisé mathématiquement cet oracle et démontré qu'en renvoyant un simple message d'erreur, il permet à un attaquant de déterminer si un message déchiffré m se situe dans l'intervalle strict $2B \leq m < 3B$.

Alors que le chapitre 3 illustre comment un choix imprudent de paramètres (un exposant secret d trop petit) pouvait compromettre le système RSA via l'attaque de Wiener, nous abordons ici une attaque différente. Les travaux de Daniel Bleichenbacher, ont prouvé que même avec des clés RSA d'une taille robuste et parfaitement choisies, la sécurité du système peut s'effondrer intégralement à cause d'une faille au niveau du protocole et de la gestion des erreurs.

L'attaque de Bleichenbacher, aussi appelée attaque par oracle, est une attaque adaptative à texte chiffré choisi. En exploitant itérativement l'oracle de conformité, l'attaquant multiplie le texte chiffré cible par des valeurs choisies et soumet ces nouveaux cryptogrammes à la cible. Les réponses binaires (Vrai/Faux) de l'oracle lui permettent de réduire progressivement l'espace de recherche, jusqu'à isoler et extraire le message clair d'origine sans jamais posséder la clé privée.

5.1 Phase 1 : Le masquage (Blinding)

L'objectif de cette phase initiale est de générer un point de départ valide pour l'algorithme en transformant le texte chiffré cible c en un message c_0 dont le déchiffrement m_0 est conforme au standard PKCS #1 v1.5.

Données et hypothèses

Dans cette phase, nous définissons les éléments à la disposition de l'attaquant :

- **Données connues** : Le texte chiffré cible c , la clé publique (n, e) , et la constante $B = 2^{8(k-2)}$.
- **Accès à l'oracle** : L'attaquant dispose d'un accès à un oracle de déchiffrement $\mathcal{O}$ qui, pour tout texte chiffré x , renvoie :
 - 1 si $x^d \pmod{n}$ est PKCS-conforme (commence par les octets 00 02).
 - 0 sinon.
- **Inconnue** : Le message clair m tel que $c \equiv m^e \pmod{n}$.

Algorithme de masquage

L'attaquant cherche un entier s_0 (facteur de masquage) qui lui permettra de confiner le message dans un intervalle de départ connu.

1. **Initialisation** : Si l'oracle confirme que le message cible est déjà conforme ($\mathcal{O}(c) = 1$), l'attaquant peut choisir $s_0 = 1$ et $c_0 = c$.
2. **Recherche aléatoire** : Sinon, l'attaquant choisit des entiers s_0 aléatoires et calcule par homomorphisme multiplicatif :

$$c_0 \equiv c \cdot (s_0^e) \pmod{n} \quad (5.1)$$

3. **Validation** : L'opération est répétée jusqu'à ce que l'oracle valide c_0 ($\mathcal{O}(c_0) = 1$).

Résultat de la phase et analyse de complexité

À l'issue de cette phase, l'attaquant a trouvé un couple (s_0, c_0) tel que le message déchiffré $m_0 \equiv m \cdot s_0 \pmod{n}$ satisfait l'inégalité fondamentale :

$$2B \leq m_0 \leq 3B - 1 \quad (5.2)$$

L'attaquant a ainsi transformé le problème initial en la recherche d'un message m_0 confiné dans un unique intervalle $M_0 = \{[2B, 3B - 1]\}$.

Pour mesurer l'efficacité de cette étape, comparons l'espace de recherche :

- **Approche naïve** : Sans information, le message m appartient à l'ensemble des entiers $[0, n - 1]$, soit un espace de taille 2^{1024} pour un module standard.
- **Espace après Phase 1** : L'intervalle M_0 possède une largeur d'environ $B = 2^{8(k-2)}$, soit 2^{1008} pour $k = 128$.

Pour évaluer la complexité de cette première phase, il convient d'estimer le nombre moyen de requêtes nécessaires avant d'obtenir une réponse positive de l'oracle. En choisissant s_0 aléatoirement, on modélise le produit $m \times s_0 \pmod n$ comme une variable uniformément distribuée sur $[0, n - 1]$.

Soit p la probabilité qu'un tirage place le message masqué exactement dans l'intervalle conforme $[2B, 3B - 1]$.

$$p \approx \frac{\text{taille } M_0}{\text{taille } \mathbb{Z}/n\mathbb{Z}} \approx \frac{B}{n}$$

Puisque l'attaquant répète ces tirages aléatoires et indépendants jusqu'au premier succès, le nombre d'essais suit une **loi géométrique** de paramètre p . L'espérance mathématique de cette loi, qui correspond au nombre moyen de requêtes, est donnée par :

$$E(X) = \frac{1}{p} = \frac{n}{B}$$

Où X modélise le nombre total d'essais nécessaires pour obtenir le tout premier succès.

Pour une clé standard de $k = 128$ octets, sachant que $n \approx 2^{1024}$ et $B = 2^{1008}$, l'espérance est de :

$$E(X) = \frac{n}{B} \approx \frac{2^{1024}}{2^{1008}} = 2^{16} = 65\,536 \text{ requêtes.}$$

Bien que l'intervalle M_0 reste vaste, l'attaquant a réduit l'incertitude de manière significative au prix d'environ 65 536 requêtes en moyenne. Surtout, il a ancré le message dans une structure mathématique exploitable. C'est cette connaissance précise qui permet de passer d'une recherche à complexité exponentielle à une réduction d'intervalles adaptative de complexité polynomiale.

5.2 Phase 2 : Recherche de messages conformes

L'objectif de cette phase est d'affiner la connaissance de l'attaquant sur le message m_0 en exploitant l'homomorphisme multiplicatif du RSA. En multipliant le texte chiffré par différents facteurs s_i^e , l'attaquant « déplace » le message clair dans l'anneau $\mathbb{Z}/n\mathbb{Z}$. Chaque fois que l'oracle confirme qu'un nouveau texte chiffré c_i est conforme, cela impose une nouvelle contrainte mathématique sur m_0 , permettant ainsi de diviser et d'éliminer des plages de valeurs incorrectes.

Définitions des variables et notations

Pour garantir la rigueur de l'algorithme, nous définissons les termes suivants :

- M_{i-1} : Il s'agit de l'ensemble des intervalles $[a, b]$ dans lesquels le message m_0 est susceptible de se trouver après l'étape $i - 1$. Au début de l'attaque ($i = 1$), cet ensemble est réduit à un seul intervalle $M_0 = \{[2B, 3B - 1]\}$.
- s_i : Le multiplicateur entier (ou facteur de recherche) que l'attaquant tente de trouver à l'étape i pour provoquer une nouvelle réponse positive de l'oracle.
- r_i : Un entier représentant le nombre de fois que le produit $m_0 \cdot s_i$ a dépassé le module n (le nombre de réductions modulaires effectuées).

2.a : Recherche du premier multiple ($i = 1$)

L'attaquant cherche le plus petit entier s_1 tel que le produit $m_0 \cdot s_1$ soit strictement supérieur au module n , et que le message déchiffré correspondant soit PKCS-conforme.

Pour que le message $m_0 \cdot s_1 \pmod{n}$ soit conforme, il doit être compris dans l'intervalle $[2B, 3B - 1]$. Or, nous savons que m_0 est déjà dans $[2B, 3B - 1]$. Pour que le produit « dépasse » le module n et revienne dans la zone conforme (en faisant au moins un tour), il faut que :

$$m_0 \cdot s_1 \geq n + 2B \quad (5.3)$$

Puisque la valeur maximale de m_0 est $3B - 1$, nous obtenons :

$$s_1 \geq \frac{n + 2B}{m_0} > \frac{n}{3B} \quad (5.4)$$

C'est pourquoi la recherche débute par :

$$s_1 = \left\lceil \frac{n}{3B} \right\rceil \quad (5.5)$$

2.b : Recherche avec plusieurs intervalles ($i > 1$)

Si l'ensemble M_{i-1} contient plusieurs intervalles disjoints ($|M_{i-1}| > 1$), l'incertitude sur la position de m_0 est trop grande pour prédire s_i . L'attaquant procède alors à une recherche linéaire exhaustive :

$$s_i = s_{i-1} + 1, s_{i-1} + 2, \dots \quad (5.6)$$

L'objectif ici est de trouver n'importe quelle valeur s_i validée par l'oracle afin de pouvoir, lors de la Phase 3, réduire le nombre d'intervalles contenus dans M_i .

2.c : Recherche avec un intervalle unique (Accélération)

Lorsque l'ensemble des solutions M_{i-1} ne contient plus qu'un seul intervalle $[a, b]$, l'attaquant bascule vers une méthode prédictive. L'incertitude étant réduite, il est possible de calculer mathématiquement les prochaines valeurs à tester.

- **Calcul du nombre de tours (r_i)** : Pour garantir une convergence rapide a chaque succès la longueur de l'intervalle est divisé par deux, c'est à dire que l'attaquant force le nouveau multiplicateur à doubler par rapport au précédent Ce qui donne :

$$s_i \geq 2s_{i-1}$$

On sait que pour retomber dans la zone conforme $[2B, 3B - 1]$ avec un intervalle restreint à $[a, b]$, donc la plus petite valeur possible pour s_i est :

$$s_i \geq \frac{2B + r_i n}{b}$$

Donc

$$\frac{2B + r_i n}{b} \geq 2s_{i-1}$$

Ce qui nous donne la borne inférieure pour r_i :

$$r_i \geq \frac{2 \cdot b \cdot s_{i-1} - 2B}{n}$$

- **Calcul des bornes de (s_i)** :

À partir du nombre de tours r_i l'attaquant peut restreindre le nombre des valeurs de s_i a tester.

On a l'inéquation fondamentale de conformité :

$$2B \leq m_0 s_i - r_i n < 3B$$

ce qui nous ramène vers l'inégalité suivante :

$$\frac{2B + r_i n}{b} \leq m_0 s_i < \frac{3B + r_i n}{a}$$

On sait que le message m_0 est borné par a et b , donc :

$$\frac{2B + r_i n}{b} \leq s_i < \frac{3B + r_i n}{a}$$

Cette approche stratégique permet de doubler la précision de la connaissance sur m_0 à chaque succès, fonctionnant de manière identique à une recherche dichotomique.

5.3 Phase 3 : Réduction de l'ensemble des intervalles

Après avoir identifié un nouveau multiple conforme s_i lors de la phase précédente, l'attaquant met à jour l'ensemble des solutions potentielles pour le message m_0 . Cette étape repose sur l'intersection des contraintes accumulées.

L'utilisation de l'indice i souligne le caractère **adaptatif** de l'attaque. Si l'on utilisait uniquement s_1 , nous n'aurions qu'une seule contrainte statique. En augmentant s_i à chaque itération, l'attaquant « zoome » sur le message :

- s_1 fournit une première réduction grossière de l'espace.
- Chaque s_i suivant est choisi beaucoup plus grand que le précédent (notamment en phase 1.2.c), ce qui permet de diviser les intervalles restants en segments de plus en plus fins.

Existence et rôle de l'entier r

Pour chaque s_i validé par l'oracle, nous savons que $m_0 s_i \pmod{n}$ est conforme, donc compris dans $[2B, 3B - 1]$. Par définition du modulo, il existe nécessairement un entier r tel que :

$$2B \leq m_0 s_i - rn \leq 3B - 1 \quad (5.7)$$

Cet entier r représente le nombre de multiples du module n que le produit $m_0 s_i$ a « franchis » avant de retomber dans la zone conforme. Mathématiquement, r existe car tout entier X peut s'écrire sous la forme $X = qn + r'$ avec $0 \leq r' < n$. Ici, rn est la partie multiple de n , et le reste r' est précisément la valeur conforme validée par l'oracle.

Proposition 1.3.1. Soit s_i un multiplicateur tel que le produit $m_0 s_i \pmod{n}$ est conforme. Sachant que le message masqué m_0 appartient à l'ensemble M_{i-1} (qui peut être une union de plusieurs intervalles disjoints $\bigcup[a, b]$), alors pour chaque sous-intervalle $[a, b]$, l'entier r satisfait les inégalités suivantes :

$$\left\lceil \frac{as_i - 3B + 1}{n} \right\rceil \leq r \leq \left\lfloor \frac{bs_i - 2B}{n} \right\rfloor \quad (5.8)$$

Justification :

D'après l'inégalité de conformité validée par l'oracle (Équation 1.7), nous avons :

$$2B \leq m_0 s_i - rn \leq 3B - 1$$

Ce qui nous permet de retrouver les bornes suivantes pour r :

$$\frac{m_0 s_i - 3B + 1}{n} \leq r \leq \frac{m_0 s_i - 2B}{n}$$

On remplace m_0 par sa valeur minimale dans la borne inférieure qui est a , et sa valeur maximale dans la borne supérieure et ce qui nous donne l'inégalité suivante :

$$\frac{as_i - 3B + 1}{n} \leq r \leq \frac{bs_i - 2B}{n} \quad (5.9)$$

Puisque l'ensemble M_{i-1} peut contenir plusieurs intervalles possibles pour m_0 , l'attaquant doit évaluer ces bornes pour chaque intervalle candidat $[a, b]$ (où $a \leq m_0 \leq b$) :

Ainsi, si l'ensemble M_{i-1} contient n intervalles distincts, l'attaquant calculera n plages de valeurs possibles pour r , garantissant de n'oublier aucune solution.

convergence de l'algorithme

L'attaquant calcule le nouvel ensemble M_i par intersection :

$$M_i = \bigcup_{a,b,r} \left([a, b] \cap \left[\left\lceil \frac{2B + rn}{s_i} \right\rceil, \left\lfloor \frac{3B - 1 + rn}{s_i} \right\rfloor \right] \right) \quad (5.10)$$

Comme le démontre Bleichenbacher [7], l'ensemble M_i finit par se réduire à un intervalle unique : au début, M_i peut contenir plusieurs segments disjoints car plusieurs valeurs de r peuvent satisfaire la condition de conformité.

Cependant, à mesure que s_i augmente, la largeur des nouveaux intervalles candidats devient plus petite que l'écart entre deux valeurs successives de r . Mathématiquement, l'intersection finit par ne retenir qu'un seul segment cohérent avec toutes les itérations précédentes.

En Phase 2.c, l'attaquant choisit s_i de manière stratégique pour qu'il soit au moins deux fois plus grand que le précédent ($s_i \geq 2s_{i-1}$). En observant la formule d'intersection de la Phase 3, on remarque que pour un entier r donné, la longueur L du nouvel intervalle candidat est :

$$L \approx \frac{3B - 1 + rn}{s_i} - \frac{2B + rn}{s_i} = \frac{B - 1}{s_i}$$

Cette équation démontre que la longueur de l'intervalle résultant dépend du multiplicateur s_i (Plus s_i est grand plus la longueur de l'intervalle est réduite). Par conséquent, l'opération consistant à doubler s_i à chaque itération divise mécaniquement la largeur de l'espace de recherche par deux ($L_i \leq \frac{L_{i-1}}{2}$). C'est cette propriété qui donne à l'attaque sa progression logarithmique, similaire à une recherche dichotomique.

Dès que $M_i = [x, x]$ c'est à dire que la longueur de l'intervalle est égale à 1 $L = 1$, alors l'algorithme converge et le message est trouvé $x = m_0$.

5.4 Phase 4 : Extraction du message clair

La phase finale de l'attaque intervient lorsque le processus itératif de réduction d'intervalles décrit dans les phases précédentes parvient à sa conclusion. L'algorithme s'arrête dès que l'ensemble des solutions potentielles M_i est réduit à un unique intervalle ne contenant qu'un seul entier .

Calcul du message masqué

À l'issue de la dernière itération, nous obtenons un intervalle de la forme $M_i = [x, x]$. Cet entier x correspond exactement à la valeur du message masqué m_0 tel que :

$$m_0 = x \quad (5.11)$$

Inversion du masquage (Unblinding)

Pour retrouver le message original m correspondant au texte chiffré initial c , l'attaquant doit inverser l'opération de masquage effectuée lors de la Phase 1. On a :

$$\begin{aligned} c_0 &\equiv c \cdot (s_0^e) \pmod{n} \\ c_0^d &\equiv (c \cdot s_0^e)^d \pmod{n} \\ m_0 &\equiv m \cdot s_0 \pmod{n} \\ m &\equiv m_0 \cdot s_0^{-1} \pmod{n} \end{aligned}$$

puisque $m_0 = x$ on peut retrouver m en utilisant l'inverse multiplicatif du facteur de masquage s_0 modulo n , le message original est calculé par :

$$m \equiv x \cdot s_0^{-1} \pmod{n} \quad (5.12)$$

finale

Une fois l'entier m retrouvé, il suffit de le convertir en une chaîne d'octets pour obtenir le bloc EB (Encryption Block). Conformément au standard PKCS#1 v1.5, le message utile M est extrait en retirant les octets de formatage obligatoires (00 02), la chaîne de remplissage PS et l'octet séparateur 00 .

L'attaque démontre ainsi qu'avec un accès répété à un simple oracle binaire de conformité, la sécurité du chiffrement RSA est totalement compromise, permettant le déchiffrement complet du secret sans avoir accès à la clé privée d .

Conclusion

Ce projet nous a permis d'explorer le cryptosystème RSA et d'avoir une compréhension plus avancée et profonde de ce dernier, et cela sur deux points majeures : d'une part, sa robustesse mathématique liée à la factorisation, et d'autre part, la fragilité de ses implémentations concrètes.

Dans un premier temps, nous avons établi les bases du système en nous appuyant sur l'arithmétique modulaire et les théorèmes fondamentaux de Fermat et d'Euler. Cette étude a mis en lumière l'élégance de la construction RSA, mais aussi l'importance d'optimisations pratiques, comme l'utilisation du théorème des restes chinois (CRT), qui permet d'accélérer significativement le processus de déchiffrement.

Cependant, nous avons démontré que cette puissance mathématique est extrêmement sensible au choix des paramètres de sécurité. L'analyse de la taille et de la proximité des nombres premiers, ainsi que l'étude détaillée de l'attaque de Wiener, nous ont rappelé qu'une quête excessive d'efficacité, notamment par l'usage d'exposants secrets trop courts peut conduire à un effondrement total de la confidentialité via l'analyse des fractions continues.

Enfin, l'analyse du standard PKCS#1 v1.5 et de l'attaque de Bleichenbacher a mis en évidence une vulnérabilité d'ordre protocolaire très critique. Nous avons vu qu'un simple message d'erreur, agissant comme un oracle, transforme un système théoriquement sûr en un canal de fuite d'information. Cette attaque prouve que la sécurité ne réside pas uniquement dans la taille des nombres premiers.

En conclusion, ce projet souligne que la cryptographie moderne est un équilibre fragile entre théorie et pratique. Si le cœur mathématique du RSA reste robuste, sa survie dans les systèmes actuels dépend d'une implémentation irréprochable et de l'adoption de standards plus résistants aux attaques par canaux auxiliaires, tels que le RSA-OAEP (*Optimal Asymmetric Encryption Padding*). Ce projet nous aura ainsi permis de comprendre que dans le monde de la cryptographie et la cybersécurité, le moindre bit d'information fuité peut casser tout un système.

Bibliographie

- [1] R. L. Rivest, A. Shamir et L. Adleman, « A Method for Obtaining Digital Signatures and Public-Key Cryptosystems », *Communications of the ACM*, vol. 21, n° 2, pp. 120–126, 1978.
- [2] A. Dujella, « Continued Fractions and RSA with Small Secret Exponent », *Preprint arXiv :cs/0402052v1*, fév. 2004.
- [3] A. Kraus, *Cours de cryptographie MM029 - Chapitre I : Arithmétique*, Université Pierre et Marie Curie (UPMC), 2009. [Support de cours sur les bases arithmétiques]
- [4] A. Nitaj, « La cryptanalyse de RSA », Laboratoire de Mathématiques Nicolas Oresme, Université de Caen, 2009. <https://nitaj.users.lmno.cnrs.fr/CryptRSA.pdf>
- [5] S. Vinatier, *Support de cours : Introduction a la cryptologie*, Université de Limoges, 2025.
- [6] N. Douziech et T. Jannaud, « Attaque de clés RSA par la méthode de Wiener », École Polytechnique, Rapport de projet INF582, mars 2008.
- [7] D. Bleichenbacher, « Chosen ciphertext attacks against protocols based on the RSA encryption standard PKCS#1 », *Advances in Cryptology — CRYPTO '98*, Springer, pp. 1–12, 1998.
- [8] B. Kaliski, « PKCS #1 : RSA Encryption Version 1.5 », *Request for Comments (RFC) 2313*, IETF, mars 1998. <https://www.rfc-editor.org/info/rfc2313>

.1 Implémentation d'une attaque naïve et l'attaque de Wiener

```
1#!/usr/bin/python3
2import math
3import time
4import sys
5
6def egcd(a, b): #Algorithme d'euclide étendu
7    x,y, u,v = 0,1, 1,0
8    while a != 0:
9        q, r = b//a, b%a
10        m, n = x-u*q, y-v*q
11        b,a, x,y, u,v = a,r, u,v, m,n
12    gcd = b
13    return gcd, x, y #Nous retournes le pgcd ainsi que les
14        coefficients de Bezout
15
16def modinv(a, m):# calcul de l'inverse modulaire
17    gcd, x, y = egcd(a, m)
18    if gcd != 1:
19        return None # L'inverse n'existe pas
20    return x % m
21
22def lpowmod(x, y, n):#x^y mod n
23    result=1
24    while y>0:
25        if y&1>0:
26            result = (result*x)%n
27        y>>=1
28        x = (x*x)%n
29    return result
30
31# Les deux fonctions qui suivront seront utilisées pour l'
32# attaque naïve
33def Factorisation(n): # Cette fonction factorise n par la méthode naïve
34    if n%2==0:
35        return 2, n/2
36    for i in range(3, math.isqrt(n)+1, 2): # ça va jusqu'à
37        racine(n)+1 par un pas de 2 à partir de 3 pour éviter les
38        multiples de 2
39        if n%i==0:
40            return i, n//i
41    return None, None
42
43def attaque_naive(n,e,c): # Fonction de l'attaque naïve
44    print(f"Début de l'attaque naïve sur ({n, e})")
45    start_time=time.time()
46    p,q= Factorisation(n) # Fait appel à la fonction de
47    factorisation
48    if p is None:
49        print("Les valeurs introduites ne peuvent pas représenter un système RSA") # si elle affiche cela il y a des
```

```

erreurs dans les valeurs choisis
44 else:
45     end_time=time.time()
46     temps_calcul=end_time-start_time #pour voir combien a
durer l'attaque naive
47     print(f"Factorisation réussie p={p}, q={q}")
48     phi_n=(p-1)*(q-1)
49     print(f" La Factorisation a pris {temps_calcul:.3f}
secondes")
50     try:
51         d=modinv(e,phi_n)
52         m=lpowmod(c,d,n)
53         return p,q,d,m
54     except:
55         print("e ne peut pas etre la clé publique de ce
système") #c'est a dire que e n'est pas premier avec e donc
erreur dans les valeurs choisis
56         return None
57 #Les 4 fonctions qui suivront seront utilisées pour l'attaque
de Wiener
58 def fractions_continues(e,n): # Calcul des fractions continues
59     quotients=[]
60     num, denom =e,n
61     while denom!=0:
62         q = num // denom
63         r = num % denom
64         quotients.append(q)
65         num,denom=denom,r
66     return quotients # Nous renvoie une liste avec les q_i
67
68 def convergentes(quotients):#Calcul des convergentes
69     k=[]
70     d=[] # initialisations des d_i et k_i
71     if len(quotients)==0:# au cas ou la liste quotients est
vide ce qui n'arrivera pas si les valeurs choisis sont
acceptables pour un systeme RSA
72         return []
73     k.append(quotients[0])
74     d.append(1)# On initialise k_0 et d_0
75     if len(quotients)==1:
76         return list(zip(k,d))# Pareil on est pas censé arrivé
la si le systeme a des valeurs convenables
77     k.append(quotients[1]*quotients[0]+1)
78     d.append(quotients[1])
79     for i in range (2,len(quotients)):# Cette boucle continue a
calculer les convergentes suivant le projet
80         val_k = quotients[i] * k[i-1] + k[i-2]
81         val_d = quotients[i] * d[i-1] + d[i-2]
82         k.append(val_k)
83         d.append(val_d)
84     return zip(k,d)# nous retourne une liste de convergentes
85
86 def resolution_quadratique(a,b,c): # cette fonction est
specialement pour l'étape 3 de l'algorithme de Wiener pour

```

```

    calculer p et q
87    delta = b*b -4*a*c
88    if delta<0:
89        return None
90    sqrt_delta=math.isqrt(delta)
91    if sqrt_delta*sqrt_delta != delta: # Verifie que le
    discriminant est un carré parfait
92        return None
93    x1=(-b+sqrt_delta)/(2*a)
94    x2=(-b-sqrt_delta)/(2*a)
95    return x1,x2
96
97 def Attaque_Wiener(n,e,c):# et celle ci est l'attaque de Wiener
98     print(f"Début de l'attaque Wiener sur (n,e)={n,e}")
99     start_time=time.time()
100     Q=fractions_continues(e,n)
101     C=convergentes(Q)
102     for i ,j in C:
103         if i== 0: #pour eviter la division par 0 au debut
104             continue
105         if (e*j-1) % i !=0: #Si phi n'est pas un entier on
    passe puisque il n'est pas acceptable
106             continue
107         phi_valide=(e*j-1)//i
108         racines=resolution_quadratique(1,-n+phi_valide-1,n)
109         try:
110             if racines:
111                 p,q=racines
112                 if p*q==n:
113                     end_time=time.time()
114                     temps_calcul=end_time-start_time
115                     print(f"Attaque de Wiener réussie ")
116                     print(f"Temps de résolution {temps_calcul
    :.10f} ")
117                     m=lpowmod(c,j,n)
118                     return p,q,j,m
119             except:
120                 print("Echec de l'attaque de wiener d n'est pas
    assez petit")
121
122
123 # Ici c'est l'application on a chosi des valeurs adaptés pour
    que 'attaque naive ne prenne pas beaucoup de temps et qu'on
    puisse l'appliquer sur nos PC
124 n = 2250001506000001003
125 e=556931065099009901 # son inverse d est 101 donc il est assez
    petit pour qu'on puisse appliquer l'attaque de Wiener
126 c = lpowmod(12345, e, n)# Calcul d'un texte chiffré a partir d'
    un message clair 12345
127 #Démarrage de l'attaque naive
128 print("-----Attaque naive-----")
129 p,q,d,m=attaque_naive(n,e,c)
130 phi_n=(p-1)*(q-1)
131 print(f"La fonction indicatrice d'euler est egale a {phi_n}")

```

```

132
133 print(f"La clé privé est trouvé d={d}")
134 print(f"Le message est retrouver {m}")
135 print("-----Fin de l'attaque naive
      -----")
136 # Démarrage Attaque de wiener
137 print("-----Attaque de Wiener
      -----")
138 p1,q1,d1,m1=Attaque_Wiener(n,e,c)
139 phi_n1=(p1-1)*(q1-1)
140 print(f"La fonction indicatrice d'euler est egale a {phi_n1}")
141
142 print(f"La clé privé est trouvé d={d1}")
143 print(f"Le message est retrouver {m1}")
144 print("-----Fin Attaque de Wiener
      -----")

```

.2 Résultat d'application du code

```

karim@PCKarim:/mnt/c/users/rahma/desktop/pir$ python3 Attaque.py
-----Attaque naive-----
Début de l'attaque naive sur ((2250001506000001003, 556931065099009901))
Factorisation réussie p=1500000001, q=1500001003
La Factorisation a pris 90.776 secondes
La fonction indicatrice d'euler est egale a 2250001503000000000
La clé privé est trouvé d=101
Le message est retrouver 12345
-----Fin de l'attaque naive-----
-----Attaque de Wiener-----
Début de l'attaque Wiener sur (n,e)=(2250001506000001003, 556931065099009901)
Attaque de Wiener réussie
Temps de résolution 0.0001289845
La fonction indicatrice d'euler est egale a 2250001503000000000
La clé privé est trouvé d=101
Le message est retrouver 12345
-----Fin Attaque de Wiener-----

```

FIGURE 1 – Résultat d'application du code

.3 Implémentation de l'attaque par oracle

```
1  #!/usr/bin/python3
2  import os
3  import random
4  from sympy import isprime
5  #=====
6  #Fonctions pour les calculs mathématiques de RSA
7  #=====
8  #fonction génératrice de nombres premiers de manières optimisés
9  def generation_nombre(taille):
10
11      while True:
12          chiffres = [random.choice('123456789')]
13          for i in range(taille-2):
14              chiffres.append(random.choice('0123456789'))
15          chiffres.append(random.choice('1379'))
16          nombre=int("".join(chiffres))
17          if isprime(nombre):
18              return nombre
19          else:
20              while not isprime(nombre) :
21                  chiffres.pop(0)
22                  chiffres.pop()
23                  chiffres.append(random.choice('0123456789'))
24                  chiffres.append(random.choice('1379'))
25                  if chiffres[0]=='0':
26                      chiffres[0]=random.choice('123456789')
27                  nombre=int("".join(chiffres))
28              return nombre
29  #Une fonction qui calcule le pgcd et les coefficients de Bezout
30
31  def egcd(a, b):
32      x,y, u,v = 0,1, 1,0
33      while a != 0:
34          q, r = b//a, b%a
35          m, n = x-u*q, y-v*q
36          b,a, x,y, u,v = a,r, u,v, m,n
37          gcd = b
38      return gcd, x, y
39  #Calcul de l'inverse modulaire
40  def modinv(a, m):
41      gcd, x, y = egcd(a, m)
42      if gcd != 1:
43          return None
44      return x % m
45  #Fonction pour la génération des clés RSA
46  def generation_cles(taille):
47      p=generation_nombre(taille)
48      q=generation_nombre(taille)
49      while q==p:
50          q=generation_nombre(taille)
51      n=p*q
```

```

51 phi_n= (p-1)*(q-1)
52 while True:
53     e=random.randint(3,phi_n-1)
54     gcd,x,y=egcd(e,phi_n)
55     if gcd==1:
56         break
57 d=modinv(e,phi_n)
58 return(n,e,d)
59
60 # Cette fonction représente L'oracle renvoie True si le message
    est PKCS-conforme, False sinon
61 def oracle(c_int):
62
63     m_int = pow(c_int, d, n) #Déchiffrement RSA
64     try:
65         #Conversion de l'entier en blocs de K octets
66         m_bytes = m_int.to_bytes(k, byteorder='big')
67     except OverflowError:
68         #L'entier est très grand pour ce module n
69         return False
70
71     # Vérification stricte du standard : 00 || 02 || Padding ||
        00 || Message
72     if m_bytes[0] != 0x00 or m_bytes[1] != 0x02:
73         return False
74     try:
75         #Recherche de l'octet séparateur nul 00
76         idx = m_bytes.index(0x00, 2)
77     except ValueError:
78         #Pas d'octet séparateur trouvé
79         return False
80     if idx - 2 < 8: # Le padding doit faire au moins 8 octets
81         return False
82
83     return True #Si tout est valide la fonction retourne True
84
85 #La fonction qui chiffre suivant le standard PKCS
86 def pkcs1_v15_chiffrement(message_clair: bytes, e: int, n: int)
    -> bytes:
87
88     # Calcul de k : la taille du module n en octets
89     k = (n.bit_length() + 7) // 8
90
91     # Vérification de la taille limite du message
92     if len(message_clair) > k - 11:
93         raise ValueError(f"Message trop long. Max autorisé pour
        ce module : {k - 11} octets.")
94     # Calcul de la longueur nécessaire pour le padding
95     ps_length = k - len(message_clair) - 3
96     # Génération du padding aléatoire
97     ps = bytearray()
98     while len(ps) < ps_length:
99         random_byte = os.urandom(1)
100         if random_byte != b'\x00': # On rejette les octets nuls
            ps.extend(random_byte)

```

```

101
102     # Assemblage du bloc formaté : 00 || 02 || Padding || 00 ||
    Message
103     eb = b'\x00\x02' + bytes(ps) + b'\x00' + message_clair
104     #La conversion de la chaîne d'octets en entier
105     x = int.from_bytes(eb, byteorder='big')
106
107     #Le calcul RSA
108     y = pow(x, e, n)
109     #La conversion de l'entier en chaîne d'octets
110     c = y.to_bytes(k, byteorder='big')
111
112     return c #Nous retourne le chiffré
113 # Cette fonction Caclule de la partie entière superieure
114 def div_plafond(a, b):
115     return (a + b - 1) // b
116 #Cette fonction sert a la réduction des intervalles
117 def maj_intervalles(M_prec, s):
118     M_nouv = []
119     #On parcourt tous les intervalles restants
120     for a, b_val in M_prec:
121         #Calcul des bornes de r: le nombre de tours
122         r_min = div_plafond(a * s - 3 * B + 1, n)
123         r_max = (b_val * s - 2 * B) // n
124         #Calcul des nouvelles bornes [a,b] pour m_0
125         for r in range(r_min, r_max + 1):
126             borne_inf = max(a, div_plafond(2 * B + r * n, s))
127             borne_sup = min(b_val, (3 * B - 1 + r * n) // s)
128             # si le snouveaux intervalles forment une
            intersection valide avec l'ancien on les gardes
129             if borne_inf <= borne_sup:
130                 M_nouv.append((borne_inf, borne_sup))
131     return M_nouv # Nous retourne M_i
132 #=====
133 #Début du Programme
134 #=====
135 #Choix des clés 256 bits
136 n, e, d = generation_cles(39)
137
138 # k = taille du module en octets (ici 32 octets)
139 k = (n.bit_length() + 7) // 8
140 B = 2*(8 * (k - 2))
141
142
143 # --- Préparation de la cible ---
144 message_secret = b"Cryptis" # Notre message en clair
145 c_bytes = pkcs1_v15_chiffrement(message_secret, e, n) #
    Chiffrement
146 c = int.from_bytes(c_bytes, byteorder='big') # Texte chiffré
    intercepté
147 print("[*] Message cible chiffré. Lancement de l'attaque...")
148
149 #=====
150 # PHASE 1 : BLINDING (Masquage)

```

```

151 #=====
152 print("[*] Phase 1 : Masquage du message (Blinding)...")
153
154 # On essaye d'abord s=1 pour verifier si notre c est déjà
    conforme
155 if oracle(c):
156     s0 = 1
157     c0 = c
158     print("    -> Le cryptogramme est déjà conforme (s0 = 1).")
159 else:
160     print("    -> Cryptogramme non conforme. Recherche d'un
    masque s0 aléatoire...")
161     while True:
162         # Tirage d'un s0 aléatoire entre 2 et n-1
163         s0 = random.randint(2, n - 1)
164         c0 = (c * pow(s0, e, n)) % n
165
166         # On interroge l'oracle pour ce s_0 random
167         if oracle(c0):
168             print(f"    -> Masque valide trouvé ! s0 = {s0}")
169             break
170
171 # =====
172 # PHASE 2.a : RECHERCHE DU PREMIER MULTIPLICATEUR (s1)
173 # =====
174 print("[*] Phase 2.a : Recherche de s1...")
175 s = n // (3 * B) #Le s est surement superieur a n/3B
176 #On continue à incrémenter le s jusqu'à ce que l'oracle
    retourne true
177 while not oracle((c0 * pow(s, e, n)) % n):
178     s += 1
179 print(f"[+] s1 trouvé : {s}")
180 M = [(2 * B, 3 * B - 1)]# Initialisation de M_0
181 M = maj_intervalles(M, s) #On met à jour l'intervalle grâce au
    s1 trouvé
182
183 # =====
184 # PHASES 2 & 3 : RÉDUCTION DES INTERVALLES
185 # =====
186 print("[*] Phase 2.b/c & 3 : Réduction itérative des
    intervalles...")
187 iteration = 1
188
189 while True:
190     # Condition d'arret si l'intervalle est réduit a un seul él
    ément
191     if len(M) == 1 and M[0][0] == M[0][1]:
192         x = M[0][0] # On a trouvé le message masqué (m0)
193
194         # =====
195         # PHASE 4 : UNBLINDING (Démasquage final)
196         # =====
197         # Calcul de l'inverse modulaire de s0 (s0^-1).
198         s0_inv = pow(s0, -1, n)

```

```

199
200     # Le vrai message :  $m = a * s_0^{-1} \bmod n$ 
201     m_trouve = (x * s0_inv) % n
202
203     # Extraction du texte en clair
204     m_bytes = m_trouve.to_bytes(k, byteorder='big')
205     idx = m_bytes.index(0x00, 2)
206     print(f"SUCCÈS ! Message décodé : {m_bytes[idx+1:].
decode('utf-8')}")
207     break
208     # =====
209     # PHASE 2.b : Recherche avec plusieurs intervalles
210     # =====
211     if len(M) > 1: #Ici on continue le balayage en essayant
plus de valeurs pour s
212         s += 1
213         while not oracle((c0 * pow(s, e, n)) % n):
214             s += 1
215     # =====
216     # PHASE 2.c : Recherche avec un intervalle unique (Accélé
ration)
217     # =====
218     elif len(M) == 1:
219         a_val, b_val = M[0]
220         r = div_plafond(2 * (b_val * s - 2 * B), n) #Calcul de
la borne inf pour r
221         trouve = False
222         while not trouve:
223             #Calcul des bornes pour s
224             s_min = div_plafond(2 * B + r * n, b_val)
225             s_max = (3 * B - 1 + r * n) // a_val
226             #On cherche la conformité dans cet intervalle
uniquement
227             for s_test in range(s_min, s_max + 1):
228                 if oracle((c0 * pow(s_test, e, n)) % n):
229                     s = s_test
230                     trouve = True
231                     break #Si le s trouvé on sort de la boucle
232             r += 1 #On passe au tour suivant
233
234     M = maj_intervalles(M, s) # La mise a jour de l'intervalle
avec le s trouvé
235     iteration += 1
236     print(f"        -> Itération {iteration}")

```

.4 Résultat de l'attaque par oracle

```
karim@PCKarim:/mnt/c/users/rahma/desktop/Attaque par oracle$ python3 Attaque_par_oracle.py
[*] Message cible chiffré. Lancement de l'attaque...
[*] Phase 1 : Masquage du message (Blinding)...
    -> Le cryptogramme est déjà conforme (s0 = 1).
[*] Phase 2.a : Recherche de s1...
[+] s1 trouvé : 11715
[*] Phase 2.b/c & 3 : Réduction itérative des intervalles...
SUCCÈS ! Message décodé : Cryptis
    -> Itération 235
karim@PCKarim:/mnt/c/users/rahma/desktop/Attaque par oracle$
```

FIGURE 2 – Résultat d'application du code